

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

по курсу «Программирование на языке Java»

на тему «Разработка многомодульных приложений на языке Java»

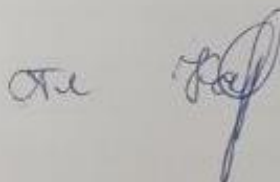
Выполнил:

студент группы 22ВВП1

Кондратьева В.И.

Принял:

Карамышева Н.С.



Пенза 2025

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Вычислительной техники

Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

профессор М. А. Митрохин

«__» ____ 20__ г

ЗАДАНИЕ

на курсовое проектирование по курсу

Программирование на языке Java
Студенту Кондратьевой Виктории Игоревне Группа 22ВВ71
Тема проекта Разработка многомодульного приложения на
языке Java

Исходные данные (технические требования) на проектирование

Разработать приложение (Экспертная система), которое должно вести диалог с пользователем: задавать вопросы и запоминать ответы пользователя. Диалог системы должен быть построен на основе утверждений, с которыми можно соглашаться или нет.

Язык программирования Java, среда разработки NetBeans.

Приложение должно обладать графическим интерфейсом и использовать следующие технологии:

1. Java Collections Framework

2. Механизм обработки исключительных ситуаций

3. Java Stream API

4. Java Multithreading

5. Сетевое взаимодействие

Объем работы по курсу

1. Расчетная часть

2. Графическая часть

Диаграммы языка UML: диаграмма вариантов использования; диаграмма классов; диаграмма деятельности; диаграмма развертывания; диаграмма последовательности

3. Экспериментальная часть

Тестирование многомодульного приложения

Срок выполнения проекта по разделам

1 В соответствии с графиком

2

3

4

5

6

7

8

Дата выдачи задания "19" февраля 2025

Дата защиты проекта "18" мая 2025

Руководитель Карамышева Н.С.

Задание получил "19" февраля 2025 г.

Студент Кондратьева В.И.

Содержание

Введение	6
1. Постановка задачи	7
2. Выбор решения	9
2.1. Графический интерфейс	9
2.2. Обработка данных	9
2.3. Сетевое взаимодействие	9
3. Описание программы	11
3.1. Клиент	11
3.2. Сервер	12
3.3. Основные модули	13
3.4. Взаимодействие компонентов	13
4. Описание способа организации пользовательского интерфейса	14
5. Описание результатов работы программы	18
Заключение	22
Список литературы	23
Приложение А. Листинг программы	24
Приложение А.1. AddAsk.java (клиент)	24
Приложение А.2. AddObject.java (клиент)	31
Приложение А.3. Answer.java (клиент)	38
Приложение А.4. Ask.java (клиент)	40
Приложение А.5. Condition.java (клиент)	41
Приложение А.6. Edit.java (клиент)	42
Приложение А.7. EditAsk.java (клиент)	65
Приложение А.8. EditCond.java (клиент)	73
Приложение А.9. EditObject.java (клиент)	79
Приложение А.10. FPSKSlaba1.java (клиент)	86
Приложение А.11. LogFrame.java (клиент)	97
Приложение А.12. Result.java (клиент)	101
Приложение А.13. Rule.java (клиент)	105

Приложение А.14. Survey.java (клиент).....	107
Приложение А.15. SurveyServer.java (сервер).....	117
Приложение В. UML – диаграммы	119
Приложение В.1. UML-диаграмма последовательности	119
Приложение В.2. UML-диаграмма вариантов использования приложения	120
Приложение В.3. UML- диаграмма развертывания.....	121
Приложение В.4. UML- диаграмма деятельности	122
Приложение В.4. UML-диаграмма классов.....	123

Введение

В современной медицине важную роль играют системы поддержки принятия решений, которые помогают врачам и пациентам в процессе диагностики заболеваний. В данной работе рассматривается разработка экспертной системы медицинской диагностики на языке Java с использованием среды разработки NetBeans. Система предназначена для анализа симптомов пациента и определения наиболее вероятных заболеваний на основе формализованных медицинских знаний.

Экспертная система представляет собой интеллектуальное приложение, которое имитирует процесс принятия решений специалистом в определенной предметной области. В данном случае предметной областью является медицинская диагностика. Система использует базу знаний, содержащую информацию о заболеваниях, их симптомах и диагностических вопросах, а также механизмы логического вывода для обработки пользовательских ответов.

Ключевой особенностью разрабатываемого приложения является клиент-серверная архитектура, где:

- Клиентская часть отвечает за взаимодействие с пользователем, включая проведение опроса и отображение результатов
- Серверная часть обеспечивает хранение и обработку истории диагностик
- База знаний хранится локально в структурированном файловом формате

Результатом работы станет полнофункциональная экспертная система, которая может быть использована в медицинских учреждениях или как самостоятельное диагностическое средство. Приложение будет обладать интуитивно понятным графическим интерфейсом и возможностью адаптации под различные предметные области за счет модификации базы знаний.

1. Постановка задачи

Разработать приложение (экспертная система) на языке Java, среда разработки NetBeans.

Приложение должно обладать графическим интерфейсом и использовать следующие технологии:

1. Java Collections Framework
2. Механизм обработки исключительных ситуаций
3. Java Stream API
4. Java Multithreading
5. Сетевое взаимодействие.

Необходимо реализовать следующие функции:

1. Загрузка и обработка базы знаний
 - 1.1. Чтение данных о заболеваниях, симптомах и вопросах из файла
 - 1.2. Валидация и парсинг структурированных данных
2. Редактирование базы знаний
 - 2.1. Добавление, изменение, удаление заболеваний (объектов)
 - 2.2. Добавление, изменение, удаление симптомов (условий)
 - 2.3. Добавление, изменение, удаление диагностических вопросов
3. Проведение диагностического опроса
 - 3.1. Последовательный вывод вопросов пользователю
 - 3.2. Обработка ответов (Да/Нет)
4. Анализ результатов
 - 4.1. Расчет вероятности заболеваний на основе ответов
 - 4.2. Определение совпадений симптомов с заболеваниями
 - 4.3. Сортировка результатов по степени вероятности
5. Визуализация результатов
 - 5.1. Вывод списка возможных диагнозов с процентной вероятностью
6. Работа с историей диагностик
 - 6.1. Логирование результатов каждого опроса
 - 6.2. Хранение истории на сервере

6.3. Просмотр предыдущих результатов

7. Настройки системы

7.1. Выбор порога вероятности для показа диагнозов

7.2. Настройка визуального оформления

7.3. Управление подключением к серверу логов

2. Выбор решения

Разрабатываемая экспертная система медицинской диагностики построена на архитектуре клиент-сервер с использованием следующих ключевых технологических решений:

2.1. Графический интерфейс

Для реализации пользовательского интерфейса выбрана библиотека Swing как стандартный компонент Java SE. Этот выбор обусловлен:

- Полной интеграцией с платформой Java без внешних зависимостей
- Широкими возможностями кастомизации внешнего вида
- Поддержкой MVC-архитектуры для компонентов
- Встроенными механизмами обработки событий

Интерфейс реализован через систему форм (JFrame), включая:

- Главное меню приложения
- Окно редактирования базы знаний
- Диагностический опросник
- Панель визуализации результатов

2.2. Обработка данных

Для работы с медицинскими знаниями используются:

- Java Collections Framework (ArrayList, HashMap) для хранения:
 - Списка заболеваний (Rule)
 - Симптомов (Condition)
 - Диагностических вопросов (Ask)
- Stream API для:
 - Фильтрации и сортировки результатов
 - Статистического анализа ответов
 - Преобразования структур данных

2.3. Сетевое взаимодействие

Для работы с историей диагностик реализовано:

- TCP-соединение через java.net.Socket

- Серверная часть на порту 12345
- Протокол обмена на основе текстовых сообщений
- Логирование результатов в файл на сервере

В приложение В.3 представлена UML-диаграмма развёртывания.

3. Описание программы

3.1. Клиент

Файл «FPSKSlaba1.java»

Основной класс приложения, отвечающий за запуск и управление графическим интерфейсом.

- `main()` – точка входа в программу, инициализирует главное окно и настраивает внешний вид.
- `jButton2ActionPerformed()` – обработчик кнопки "Загрузить данные". Открывает диалог выбора файла базы знаний, парсит данные и загружает их в систему.
- `jButton3ActionPerformed()` – обработчик кнопки "Изменить данные". Открывает окно редактирования (Edit), передавая текущие данные.
- `jButton1ActionPerformed()` – обработчик кнопки "Начать опрос". Запускает диагностический опрос (Survey), передавая список вопросов и правил.
- `jButton4ActionPerformed()` – обработчик кнопки "История". Запрашивает логи с сервера и отображает их в `LogFrame`.
- `parseRule()`, `parseCondition()`, `parseAsk()` – методы для парсинга строк из файла в объекты `Rule`, `Condition`, `Ask`.
- `sendResultsToServer()` – отправляет результаты диагностики на сервер для сохранения в лог.

Файл «Edit.java»

Окно редактирования базы знаний.

- `updateRulesTable()`, `updateConditionsTable()`, `updateAsksTable()` – обновляют таблицы с данными после изменений.
- `jButton1ActionPerformed()` – открывает `AddObject` для добавления нового заболевания.
- `jButton2ActionPerformed()` – открывает `EditObject` для редактирования выбранного заболевания.

- `jButton4ActionPerformed()` – открывает `EditCond` для добавления/редактирования симптома.

- `jButton9ActionPerformed()` – открывает `AddAsk` для добавления нового вопроса.

- `jButton10ActionPerformed()` – открывает `EditAsk` для редактирования выбранного вопроса.

- `saveRulesToFile()` – сохраняет изменения в файл базы знаний.

Файл «Survey.java»

Окно диагностического опроса.

- `UpdateWindow()` – обновляет интерфейс, показывая текущий вопрос.

- `answerButtonClicked()` – обрабатывает ответ пользователя (Да/Нет), сохраняет выбранные симптомы.

- `findBestMatches()` – анализирует ответы, вычисляет вероятность заболеваний и выводит результаты в `Result`.

- `getUserConditionIds()` – формирует список симптомов на основе ответов.

- `calculateMatchPercentage()` – вычисляет процент совпадения симптомов с заболеваниями.

Файл «Result.java»

Окно отображения результатов диагностики.

- `setResultText()` – выводит список возможных диагнозов с вероятностями.

Файл «LogFrame.java»

Окно отображения результатов диагностики.

- `setLogText()` – загружает и отображает логи с сервера.

3.2. Сервер

Файл «SurveyServer.java»

Серверная часть для хранения и обработки логов.

- `main()` – запускает сервер на порту 12345.
- `handleClient()` – обрабатывает входящие соединения.
- Принимает команду `GET_LOGS` и отправляет содержимое файла `results.txt`.
- Принимает результаты диагностики и сохраняет их в лог.
- `sendLogFile()` – читает файл логов и отправляет его клиенту.

3.3. Основные модули

- Графический интерфейс (`FPSKSlab1`, `Edit`, `Survey`, `Result`, `LogFrame`) – обеспечивает взаимодействие с пользователем.
- Обработка данных (`Rule`, `Condition`, `Ask`) – хранение и управление медицинскими знаниями.
- Диагностический модуль (`Survey`) – проведение опроса и анализ симптомов.
- Сетевой модуль (`SurveyServer`) – сохранение и загрузка истории диагностик.
- Логика работы с файлами – загрузка и сохранение базы знаний (`Edit`), логирование результатов (`SurveyServer`).

3.4. Взаимодействие компонентов

- Пользователь запускает программу (`FPSKSlab1`).
- Загружает базу знаний – парсинг в объекты `Rule`, `Condition`, `Ask`.
- Редактирует данные через `Edit` – изменения сохраняются в файл.
- Проходит опрос (`Survey`) – ответы анализируются, результаты выводятся в `Result`.
- Логи сохраняются на сервер (`SurveyServer`) – могут быть просмотрены в `LogFrame`.

В приложении В.4 представлена UML диаграмма классов клиентского приложения.

4. Описание способа организации пользовательского интерфейса

Главное окно программы (PSKSlaba1.java)

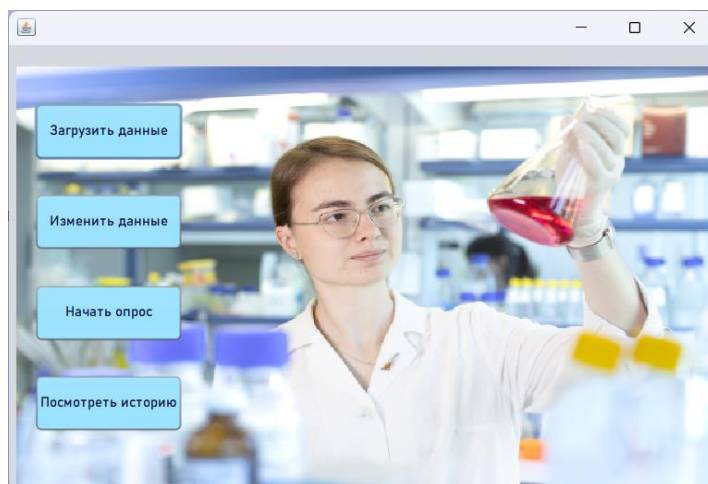


Рисунок 1 – Главное окно программы

Кнопка "Загрузить данные" – открывает диалог выбора файла с базой знаний.

Кнопка "Изменить данные" – открывает окно редактирования (Edit).

Кнопка "Начать опрос" – запускает диагностический опрос (Survey).

Кнопка "История" – отображает окно с логами диагностик (LogFrame).

Окно редактирования базы знаний (Edit.java)

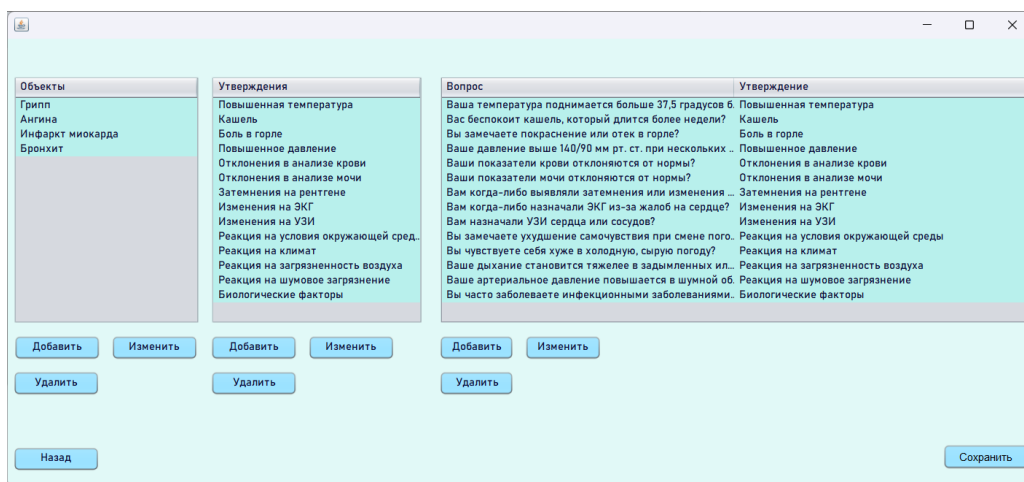


Рисунок 2 – Окно редактирования базы знаний

Таблица "Объекты" – список заболеваний (правила).

Таблица "Утверждения" – список симптомов (условий).

Таблица "Вопросы" – список диагностических вопросов.

Кнопки "Добавить/Изменить/Удалить" – управление элементами базы знаний.

Кнопка "Сохранить" – запись изменений в файл.

Редактирование данных осуществляется с помощью вспомогательных окон:

Утверждение	Относится
Повышенная температ...	☐
Кашель	☐
Боль в горле	☐
Повышенное давление	☐
Отклонения в анализе ...	☐
Отклонения в анализе ...	☐
Затемнения на рентген...	☐
Изменения на ЭКГ	☐

Рисунок 3 – Окно AddObject.java

Утверждение	Относится
Повышенная температ...	☒
Кашель	☒
Боль в горле	☒
Повышенное давление	☐
Отклонения в анализе ...	☒
Отклонения в анализе ...	☐
Затемнения на рентген...	☒
Изменения на ЭКГ	☐

Рисунок 4 – Окно EditObject.java

Название утверждения

Рисунок 5 – Окно EditCond.java

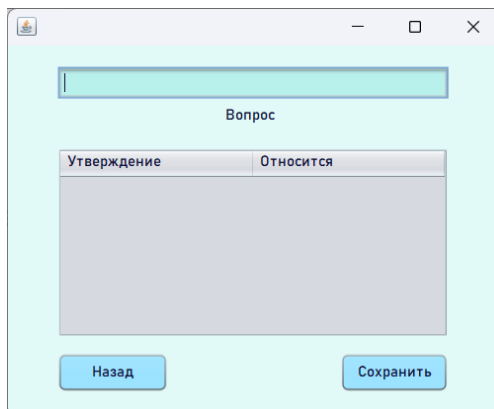


Рисунок 6 – Окно AddAsk.java

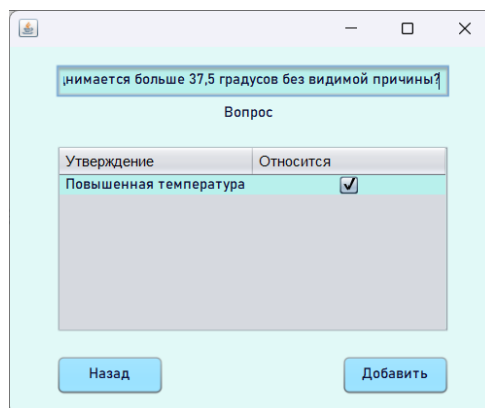


Рисунок 7 – Окно EditAsk.java

Окно диагностического опроса (Survey.java)

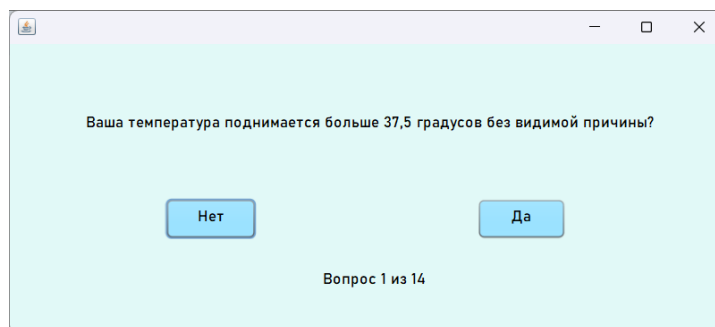


Рисунок 8 – Окно диагностического опроса

Текст вопроса – отображается в центре окна.

Кнопки "Да"/"Нет" – для выбора ответа.

Индикатор завершённости опроса.

Окно результатов (Result.java)

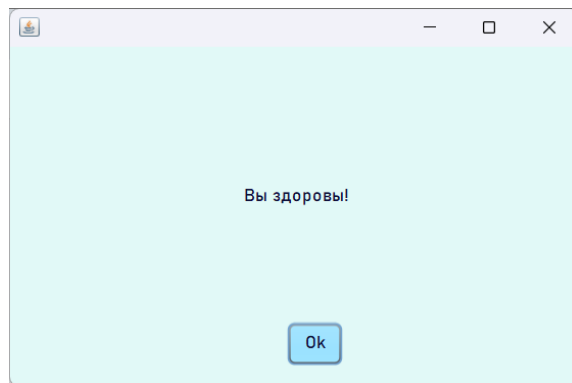


Рисунок 9 – Окно результатов

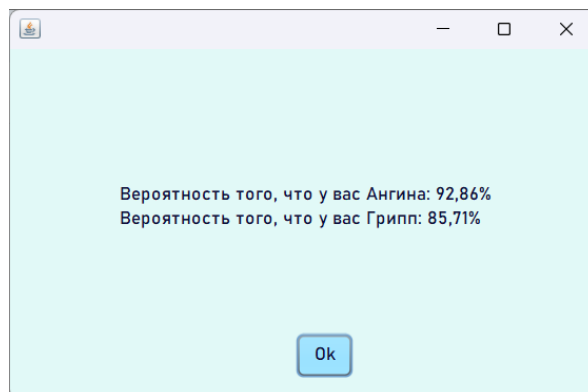


Рисунок 10 – Окно результатов

Текстовое поле – вывод результата опроса. Если нет совпадений $\geq 72\%$, то выводится "Вы здоровы", иначе выводится список соответствующих заболеваний с количеством %, отсортированный в порядке убывания.

Окно логов (LogFrame.java)

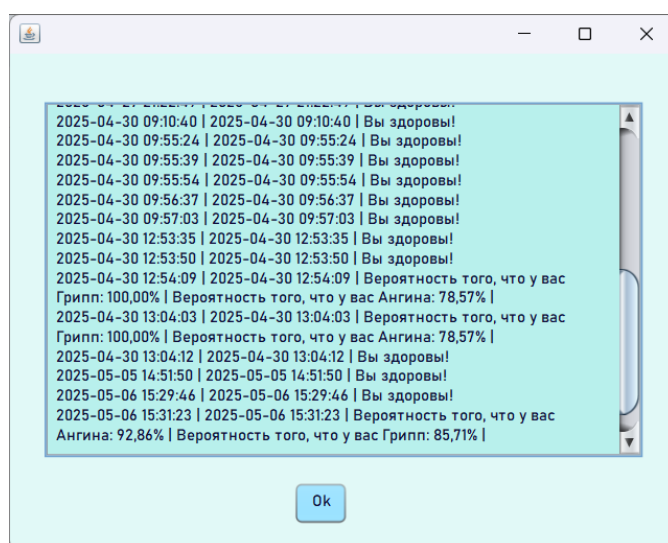


Рисунок 11 – Окно логов

Текстовое поле – отображение логов диагностик.

5. Описание результатов работы программы

Управляющие кнопки и основное окно, в котором есть три клавиши меню (Рисунок 1). Выбор конкретного пункта меню осуществляется однократным нажатием левой кнопкой мыши по нужной клавише меню. Загрузка базы знаний (верхняя клавиша меню) выполняется в случае, если база знаний была создана ранее и находится на носителе данных. При выборе этого пункта система выдает на экран оконную форму для выбора папки с имена всех файлов, содержащихся на носителе данных. Загрузка файла базы знаний инициируется клавишей «открыть» либо двойным нажатием на файл, в котором сохранена база знаний. Если база знаний в текущий момент отсутствует, пользователь должен выбрать вторую (среднюю) клавишу меню – «Изменение данных».

Общение с пользователем в интегрированной среде ЭС ведется на русском языке. Знания в ЭС заносятся на основе упрощенной продукционной модели представления знаний, причем обеспечивается возможность формулирования условий, правил, вопросов и заключений в терминах предметной области. Структура правила базы знаний ЭС включает следующие элементы:

1) ключевое слово «rule»;

2) список параметров, заключенный в круглые скобки.

Параметры правила в списке отделяются друг от друга “;” и содержат следующую информацию:

параметр 1 – id правила;

параметр 2 – название общего понятия предметной области (класса);

параметр 3 – название частного понятия предметной области (представителя), которое заключается в общее понятие (параметр 2);

параметр 4 – список условий, принадлежащих частному понятию и являющиеся критерием поиска.

Список условий продукционного правила заключается в квадратные скобки, а сами условия обозначаются числами и отделяются друг от друга запятой.

Структура условия отдельного правила, подобно самому правилу, включает ключевое слово «cond» и список параметров, заключенные в круглые скобки. Параметры условия в списке отделяются друг от друга “;” и содержат следующую информацию:

параметр 1 – id условия;

параметр 2 – формулировка условия (текст) в терминах предметной области.

Структура вопроса включает ключевое слово «ask» и список параметров, заключенные в круглые скобки. Параметры условия в списке отделяются друг от друга запятой и содержат следующую информацию:

параметр 1 – id правила;

параметр 2 – формулировка правила (текст) в терминах предметной области;

параметр 3 – номер, соответствующий связанному с вопросом условию.

Заключен в квадратные скобки

Порядок расположения в базе знаний: правила, условия, вопросы. Последний элемент базы знаний ЭС содержит имя файла на носителе данных, в котором хранится база знаний, и формулируется автоматически при создании базы.

В процессе изменения знаний пользователь осуществляет либо создание базы знаний ЭС (Рисунок 12), либо добавление необходимых правил и условий (Рисунок 2).

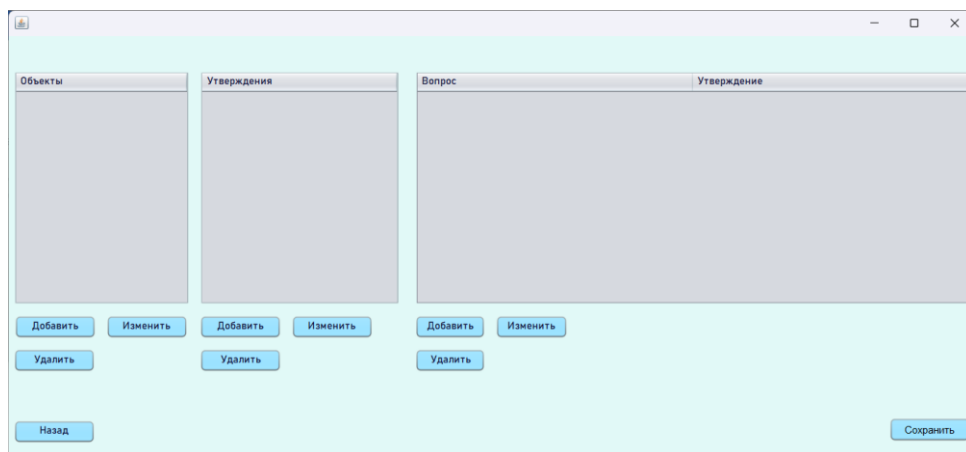


Рисунок 12 - Создание базы знаний

Для добавления новых «Объектов» в базу знаний пользователю необходимо нажать кнопку «Добавить» под таблицей «Объекты». При нажатии кнопки «Добавить» система выдает окно (Рисунок 3), в поле «Название» пользователю необходимо внести название субъекта (также, можно ответить соответствующие условия) и нажать кнопку «Добавить».

Для добавления новых «Объектов» в базу знаний пользователю необходимо нажать кнопку «Изменить» под таблицей «Объекты». Редактирование элемента базы знаний осуществляется путем повторения пользователем описанных выше действий для всех необходимых субъектов экспертной системы (Рисунок 4).

Для добавления новых «Утверждений» в базу знаний пользователю необходимо нажать кнопку «Добавить» под таблицей «Утверждения» (Рисунок 12). При нажатии кнопки «Добавить», система выдает окно (Рисунок 5), в поле «Название утверждения» необходимо внести описание субъекта и нажать кнопку «Сохранить».

Изменение следующих утверждений субъектов экспертной системы осуществляется путем повторения пользователем действий, аналогичных действиям с объектами.

Также, аналогично с предыдущими можно редактировать таблицу «Вопрос\Утверждение» (Рисунок 7). При этом можно выбрать только не занятое утверждение.

В базе знаний некоторые описания могут быть свойственны нескольким объектам, но при этом не должно быть абсолютно одинаковых субъектов, так как ЭС не сможет дать правильный ответ, такие субъекты должны отличаться хотя бы одним описанием.

Для сохранения созданной базы знаний необходимо нажать на кнопку «Сохранить».

После нажатия система выдаст окно, в котором пользователю необходимо с помощью правой кнопки мыши создать текстовый документ. Далее надо выбрать созданный файл и нажать кнопку «Открыть», таким образом, база знаний будет сохранена.

После редактирования базы знаний ЭС также необходимо выполнить аналогичные действия, реализуемым после создания знаний.

После сохранения базы знаний пользователь может начать опрос. Для этого необходимо выйти из окна «Редактирование базы знаний» при помощи кнопки «Назад». Таким образом, пользователь перешел в главное окно системы. Теперь необходимо начать опрос. После нажатия пользователю будет выведено окно опроса (Рисунок 8).

В представленном выше окне пользователю нужно отвечать на поставленные системой вопросы только «да» или «нет».

Результат опроса представлен на Рисунке 9 и Рисунке 10.

После нажатия клавиши «Ок» пользователь переходит в главную форму. При нажатии на кнопку «Посмотреть историю» пользователь может посмотреть результаты опросов за все время. Окно логов представлено на Рисунке 11.

Заключение

В процессе выполнения курсовой работы была разработана экспертная система медицинской диагностики на языке Java с использованием клиент-серверной архитектуры. Проект позволил углубить понимание принципов работы с базами знаний, механизмами логического вывода, а также сетевым взаимодействием между клиентом и сервером. Были успешно реализованы все поставленные задачи, включая загрузку и редактирование базы знаний, проведение диагностического опроса, анализ результатов и сохранение истории диагностик.

В ходе разработки были приобретены следующие навыки:

- Программирование на языке Java с использованием современных технологий, таких как Java Collections Framework, Stream API и многопоточность.
- Проектирование и реализация графического интерфейса пользователя с помощью библиотеки Swing.
- Организация клиент-серверного взаимодействия через TCP-соединения.
- Обработка и валидация данных, включая парсинг структурированных файлов.

Особое внимание уделялось модульности приложения, что позволило разделить функционал на логические компоненты: клиентскую часть для взаимодействия с пользователем, серверную часть для хранения истории диагностик и модуль обработки данных. Все компоненты системы работают стабильно, обеспечивая корректное выполнение диагностики и удобное управление базой знаний.

Разработанная экспертная система обладает потенциалом для дальнейшего развития. В будущем можно расширить функционал, добавив поддержку новых форматов данных, интеграцию с внешними API для автоматического обновления базы знаний, а также улучшить алгоритмы анализа симптомов для повышения точности диагностики.

Завершение этой курсовой работы стало важным этапом в освоении принципов разработки многомодульных приложений и заложило основу для создания более сложных и функциональных систем в будущем.

Список литературы

1. Блох Дж. Java. Эффективное программирование. — М.: Лори, 2019.
2. Эккель Б. Философия Java. — СПб.: Питер, 2022.
3. Oracle. Официальная документация по Java [Электронный ресурс]. — Режим доступа: <https://docs.oracle.com/en/java/>.
4. Stack Overflow. Сообщество разработчиков [Электронный ресурс]. — Режим доступа: <https://stackoverflow.com/>.

Приложение А. Листинг программы

Приложение А.1. AddAsk.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GuiForms/JFrame.java   to
edit this template
 */
package pskslabal;

import javax.swing.*.*;
import javax.swing.table.DefaultTableModel;
import java.util.List;
import java.util.ArrayList;
// Для работы с таблицей и её шапкой
import javax.swing.JTable;
import javax.swing.table.JTableHeader;
import java.awt.Color;
import java.awt.Font;

/**
 *
 * @author Вика
 */
public class AddAsk extends javax.swing.JFrame {
    private Edit parent;
    private List<Condition> conditions;
    private List<Ask> asks;

    /**
     * Creates new form AddAsk
     */
    public AddAsk(Edit parent, List<Ask> asks, List<Condition> conditions) {
        this.parent = parent;
        this.conditions = conditions;
        this.asks = asks;

        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
        // Получаем шапку таблицы
        JTableHeader header = jTable1.getTableHeader();

        header.setBackground(new Color(161, 227, 238));
        header.setForeground(new Color(13, 14, 57));
        header.setFont(new Font("Bahnschrift", Font.PLAIN, 12));
        fillTable(); // Заполняем таблицу утверждений
    }

    public AddAsk() {
        initComponents();
    }
}
```



```

    }

    private void fillTable() {
        DefaultTableModel model = (DefaultTableModel) jTable1.getModel();
        model.setRowCount(0); // Очищаем таблицу

        for (Condition condition : conditions) {
            boolean hasAsk = false;
            for (Ask ask : asks) {
                if (ask.getConditionIds().contains(condition.getId())) {
                    hasAsk = true;
                    break;
                }
            }
            if (!hasAsk) {
                model.addRow(new Object[]{condition.getName(), false});
            }
        }

        // Добавляем слушатель событий для чекбоксов
        jTable1.getModel().addTableModelListener(e -> {
            int selectedCount = 0;
            int selectedRow = -1;

            for (int i = 0; i < model.getRowCount(); i++) {
                Boolean isChecked = (Boolean) model.getValueAt(i, 1);
                if (isChecked != null && isChecked) {
                    selectedCount++;
                    selectedRow = i;
                }
            }

            if (selectedCount > 1) {
                JOptionPane.showMessageDialog(this, "Можно выбрать только одно утверждение!", "Ошибка", JOptionPane.ERROR_MESSAGE);
                model.setValueAt(false, selectedRow, 1); // Сбрасываем последний выбранный чекбокс
            }
        });
    }

    /**
     * This method is called from within the constructor to initialize the
    form.

    * WARNING: Do NOT modify this code. The content of this method is always
    * regenerated by the Form Editor.
    */
    @SuppressWarnings("unchecked")

```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jTextField1 = new javax.swing.JTextField();
    jLabel1 = new javax.swing.JLabel();
    jScrollPane1 = new javax.swing.JScrollPane();
    jTable1 = new javax.swing.JTable();
    jButton1 = new javax.swing.JButton();
    jButton2 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
    setBackground(new java.awt.Color(40, 167, 215));

    jTextField1.setBackground(new java.awt.Color(184, 240, 236));
    jTextField1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N

    jTextField1.setForeground(new java.awt.Color(13, 14, 57));
    jTextField1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jTextField1ActionPerformed(evt);
        }
    });

    jLabel1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jLabel1.setForeground(new java.awt.Color(13, 14, 57));
    jLabel1.setText("Вопрос");

    jTable1.setBackground(new java.awt.Color(184, 240, 236));
    jTable1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jTable1.setForeground(new java.awt.Color(13, 14, 57));
    jTable1.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {

            },
        new String [] {
            "Утверждение", "Относится"
        }
    ) {
        Class[] types = new Class [] {
            java.lang.String.class, java.lang.Boolean.class
        };
        boolean[] canEdit = new boolean [] {
            false, true
        };

        public Class getColumnClass(int columnIndex) {
            return types [columnIndex];
        }

        public boolean isCellEditable(int rowIndex, int columnIndex) {
            return canEdit [columnIndex];
        }
    });
}

```

```

        }
    });
    jTable1.setGridColor(new java.awt.Color(125, 202, 233));
    jTable1.setSelectionBackground(new java.awt.Color(255, 0, 153));

jTable1.setSelectionMode(javax.swing.ListSelectionModel.SINGLE_SELECTION);
    jTable1.setShowGrid(false);
    jScrollPane.setViewportView(jTable1);
    if (jTable1.getColumnModel().getColumnCount() > 0) {
        jTable1.getColumnModel().getColumn(0).setResizable(false);
        jTable1.getColumnModel().getColumn(1).setResizable(false);
    }

    jButton1.setBackground(new java.awt.Color(125, 202, 233));
    jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton1.setForeground(new java.awt.Color(13, 14, 57));
    jButton1.setText("Сохранить");
    jButton1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton1ActionPerformed(evt);
        }
    });

    jButton2.setBackground(new java.awt.Color(125, 202, 233));
    jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton2.setForeground(new java.awt.Color(13, 14, 57));
    jButton2.setText("Назад");
    jButton2.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton2ActionPerformed(evt);
        }
    });

    javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addGap(176, 176, 176)
                .addComponent(jLabel1))
            .addGroup(layout.createSequentialGroup()
                .addGap(40, 40, 40)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
            .addComponent(jScrollPane1,
javax.swing.GroupLayout.DEFAULT_SIZE, 317, Short.MAX_VALUE)

```

```

        .addComponent(jTextField1)
        .addGroup(layout.createSequentialGroup()
            .addComponent(jButton2,
                javax.swing.GroupLayout.PREFERRED_SIZE, 90, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(jButton1))))
        .addContainerGap(44, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addGroup(layout.createSequentialGroup()
                    .addGap(17, 17, 17)
                    .addComponent(jTextField1,
                        javax.swing.GroupLayout.PREFERRED_SIZE,
                        javax.swing.GroupLayout.DEFAULT_SIZE,
                        javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                    .addComponent(jLabel1)
                    .addGap(18, 18, 18)
                    .addComponent(jScrollPane1,
                        javax.swing.GroupLayout.PREFERRED_SIZE, 154,
                        javax.swing.GroupLayout.PREFERRED_SIZE))
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                    .addComponent(jButton1,
                        javax.swing.GroupLayout.PREFERRED_SIZE, 32, javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(jButton2,
                        javax.swing.GroupLayout.PREFERRED_SIZE, 32,
                        javax.swing.GroupLayout.PREFERRED_SIZE))
                .addContainerGap(20, Short.MAX_VALUE))
            .addGap(10, 10, 10)
        );

    pack();
} // </editor-fold>

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    this.dispose();
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    String questionText = jTextField1.getText().trim();

    if (questionText.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Введите текст вопроса!",
            "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Получаем список выбранных утверждений (но только одно!)

```

```

List<Integer> selectedConditionIds = new ArrayList<>();
DefaultTableModel model = (DefaultTableModel) jTable1.getModel();

for (int i = 0; i < model.getRowCount(); i++) {
    Boolean isChecked = (Boolean) model.getValueAt(i, 1); // Второй
столбец (чекбокс)
    if (isChecked != null && isChecked) {
        for (Condition condition : conditions) {
            if (condition.getName().equals(model.getValueAt(i, 0))) {
                selectedConditionIds.add(condition.getId());
                break;
            }
        }
        break; // Разрешаем только один выбор
    }
}

if (selectedConditionIds.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Выберите одно утверждение!",
"Ошибка", JOptionPane.ERROR_MESSAGE);
    return;
}

// Определяем ID нового вопроса
int newId = asks.stream().mapToInt(Ask::getId).max().orElse(0) + 1;

// Создаем и добавляем новый вопрос
Ask newAsk = new Ask(newId, questionText, selectedConditionIds);
asks.add(newAsk);

// Обновляем таблицу вопросов в Edit
parent.updateAsksTable();

// Закрываем окно
this.dispose();
}

private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    <editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
    *
    * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html

```

```

        */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {

javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;

                }
            }
        } catch (ClassNotFoundException ex) {

java.util.logging.Logger.getLogger(AddAsk.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (InstantiationException ex) {

java.util.logging.Logger.getLogger(AddAsk.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (IllegalAccessException ex) {

java.util.logging.Logger.getLogger(AddAsk.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(AddAsk.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        }
    }
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new AddAsk().setVisible(true);
        }
    });
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JLabel jLabel1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable jTable1;
private javax.swing.JTextField jTextField1;
// End of variables declaration
}

```

Приложение А.2. AddObject.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java   to
edit this template
 */
package pskslabal;

import javax.swing.*.*;
import java.util.List;
import javax.swing.table.DefaultTableModel;
import java.util.ArrayList;
import javax.swing.JTable;
import javax.swing.table.JTableHeader;
import java.awt.Color;    // Для работы с цветами
import java.awt.Font;    // Для работы со шрифтами

/**
 *
 * @author Вика
 */
public class AddObject extends javax.swing.JFrame {

    private Edit parent; // Добавляем переменную для хранения ссылки на Edit
    private List<Rule> rules;
    private List<Condition> conditions;

    /**
     * Creates new form AddObject
     */
    public AddObject(Edit parent, List<Rule> rules, List<Condition>
conditions) {
        this.parent = parent;
        this.rules = rules;
        this.conditions = conditions;

        printData();
        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
        JTableHeader header = jTable1.getTableHeader();
        header.setBackground(new Color(161, 227, 238)); // Пример: SteelBlue
        header.setForeground(new Color(13, 14, 57));
        header.setFont(new Font("Bahnschrift", Font.PLAIN, 12));
        fillTable(); // Заполняем таблицу
    }
}
```

```

    }

    private void printData() {
        System.out.println("Transmitted rules:");
        for (Rule rule : rules) {
            System.out.println(rule);
        }
        System.out.println("\ntransmitted conditions:");
        for (Condition condition : conditions) {
            System.out.println(condition);
        }
    }

    private void fillTable() {
        DefaultTableModel model = (DefaultTableModel) jTable1.getModel();
        model.setRowCount(0); // Очищаем таблицу перед заполнением

        for (Condition condition : conditions) {
            model.addRow(new Object[]{condition.getName(), false}); //
Добавляем в таблицу
        }
    }

    /**
     * This method is called from within the constructor to initialize the
form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        jLabel1 = new javax.swing.JLabel();
        jTextField1 = new javax.swing.JTextField();
        jScrollPane1 = new javax.swing.JScrollPane();
        jTable1 = new javax.swing.JTable();
        jButton1 = new javax.swing.JButton();
        jButton2 = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

        jLabel1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
        jLabel1.setText("Название");

```


NOI18N

```
jTextField1.setBackground(new java.awt.Color(184, 240, 236));
jTextField1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); //

jTextField1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jTextField1ActionPerformed(evt);
    }
});

jTable1.setBackground(new java.awt.Color(184, 240, 236));
jTable1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jTable1.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

        },
    new String [] {
        "Утверждение", "Относится"
    }
) {
    Class[] types = new Class [] {
        java.lang.String.class, java.lang.Boolean.class
    };
    boolean[] canEdit = new boolean [] {
        false, true
    };

    public Class getColumnClass(int columnIndex) {
        return types [columnIndex];
    }

    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
});
jScrollPane1.setViewportViewView(jTable1);
if (jTable1.getColumnModel().getColumnCount() > 0) {
    jTable1.getColumnModel().getColumn(0).setResizable(false);
    jTable1.getColumnModel().getColumn(1).setResizable(false);
}

jButton1.setBackground(new java.awt.Color(125, 202, 233));
jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton1.setText("Добавить");
jButton1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton1ActionPerformed(evt);
    }
});
```

```

    }
});

jButton2.setBackground(new java.awt.Color(125, 202, 233));
jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton2.setText("Отмена");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(172, 172, 172)
            .addComponent(jLabel1)
            .addGroup(layout.createSequentialGroup()
                .addGap(43, 43, 43)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
            .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 102,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
Short.MAX_VALUE)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.PREFERRED_SIZE, 102,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(jTextField1)
            .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE, Short.MAX_VALUE)))
            .addGap(42, Short.MAX_VALUE))
        )
);
layout.setVerticalGroup(

```

```

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(24, 24, 24)
        .addComponent(jTextField1,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(jLabel1)
            .addGap(18, 18, 18)
            .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE, 158,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(18, 18, 18)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 32, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 32,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addContainerGap(25, Short.MAX_VALUE)
    );

pack();
} // </editor-fold>

private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:

    String objectName = jTextField1.getText().trim();

    if (objectName.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Введите название объекта!",
"Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    int newId = rules.stream()
        .mapToInt(Rule::getId) // Получаем все ID
        .max() // Находим максимальный
        .orElse(0) + 1; // Если список пуст, начинаем с 1

```

```

        // Получаем список выбранных conditions (номера строк с галочкой)
        List<Integer> selectedConditionIds = new ArrayList<>();
        DefaultTableModel model = (DefaultTableModel) jTable1.getModel();

        for (int i = 0; i < model.getRowCount(); i++) {
            Boolean isChecked = (Boolean) model.getValueAt(i, 1); // Второй
            столбец (чекбокс)
            if (isChecked != null && isChecked) {
                selectedConditionIds.add(i + 1); // Индекс +1 для соответствия
            ID
            }
        }

        // Создаем новый объект Rule
        Rule newRule = new Rule(newId, "Болезни", objectName,
selectedConditionIds);
        rules.add(newRule);

        // Вызываем метод обновления таблицы в Edit
        if (parent != null) {
            parent.updateRulesTable(); // !!! Вот тут исправленный вызов !!!
        }

        // Закрываем окно
        this.dispose();
    }

    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
        this.dispose();
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
         *
         * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {

```

```

javax.swing.UIManager.setLookAndFeel(info.getClassName());
            break;
        }
    }
} catch (ClassNotFoundException ex) {

java.util.logging.Logger.getLogger(AddObject.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
    } catch (InstantiationException ex) {

java.util.logging.Logger.getLogger(AddObject.class.getName()).log(java.util.logging
Level.SEVERE, null, ex);
    } catch (IllegalAccessException ex) {

java.util.logging.Logger.getLogger(AddObject.class.getName()).log(java.util.logging
Level.SEVERE, null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(AddObject.class.getName()).log(java.util.logging
Level.SEVERE, null, ex);
    }
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {

        }
    });
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JLabel jLabel1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable jTable1;
private javax.swing.JTextField jTextField1;
// End of variables declaration
}
}

```

Приложение А.3. Answer.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
this template
 */
package pskslabal;

/**
 *
 * @author Вика
 */
public class Answer {
    private int id;
    private String name;
    private String category;
    private String condAnswerId; // Строка с номерами утверждений
    private Double percent; // Пока null

    public Answer(int id, String name, String category) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.condAnswerId = ""; // Начальное значение – пустая строка
        this.percent = null; // Пока не трогаем
    }

    public void addConditionId(int conditionId) {
        if (condAnswerId.isEmpty()) {
            condAnswerId = String.valueOf(conditionId);
        } else {
            condAnswerId += ";" + conditionId;
        }
    }

    public void printCondAnswerId() {
        System.out.println("Ответы пользователя (condAnswerId): " +
condAnswerId);
    }

    // Геттеры и сеттеры
    public int getId() {
        return id;
    }
}
```

```

public String getName() {
    return name;
}

public String getCategory() {
    return category;
}

public String getCondAnswerId() {
    return condAnswerId;
}

public Double getPercent() {
    return percent;
}

public void setPercent(Double percent) {
    this.percent = percent;
}

public String toString() {
    return "Answer{id=" + id + ", name='" + name + "', category='" +
category + "', condAnswerId='" + condAnswerId + "'}";
}
}

```

Приложение А.4. Ask.java (клиент)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package pskslabal;

import java.util.List;
import java.util.ArrayList;
/**
 *
 * @author Вика
 */
public class Ask {

    private List<Ask> asks = new ArrayList<>();
    private int id; // ID утверждения
    private String question; // Текст вопроса
    private List<Integer> conditionIds; // Список ID утверждений, связанных с
вопросом

    public Ask(int id, String question, List<Integer> conditionIds) {
        this.id = id;
        this.question = question;
        this.conditionIds = new ArrayList<>(conditionIds); // Создаем копию
списка, чтобы избежать изменений по ссылке
    }

    public int getId() {
        return id;
    }

    public String getQuestion() {
        return question;
    }

    public void setQuestion(String question) {
        this.question = question;
    }

    public List<Integer> getConditionIds() {
        return conditionIds;
    }

    public void setConditionIds(List<Integer> conditionIds) {
        this.conditionIds = new ArrayList<>(conditionIds);
    }

    @Override
    public String toString() {
        return "Ask{id=" + id + ", question='" + question + "', conditionIds=" +
conditionIds + "}";
    }
}
```


Приложение A.5. Condition.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
this template
 */
package pskslabal;

/**
 *
 * @author Вика
 */
public class Condition {
    private int id;
    private String name;

    public Condition(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public int getId() {
        return id;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Condition{" +
            "id=" + id +
            ", name='" + name + '\'' +
            '}';
    }
}
```

Приложение А.6. Edit.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GuiForms/JFrame.java   to
edit this template
 */
package pskslabal;

import javax.swing.*.*;
import java.util.List;
import javax.swing.table.DefaultTableModel;
import java.util.ArrayList; // Для работы со списками
import java.io.*; // Для работы с файлами (File, FileReader, FileWriter,
BufferedReader, BufferedWriter)
import javax.swing.JTable;
import javax.swing.table.JTableHeader;
import java.awt.Color;
import java.awt.Font;

/**
 *
 * @author Вика
 */
public class Edit extends javax.swing.JFrame {

    private List<Rule> rules;
    private List<Condition> conditions;
    private List<Ask> asks;
    private String filename;
    private String filepath;

    /**
     * Creates new form Edit
     */
    public Edit(List<Rule> rules, List<Condition> conditions, List<Ask> asks,
String filename, String filepath) {
        this.rules = rules;
        this.conditions = conditions;
        this.asks = asks;
        this.filename = filename;
        this.filepath = filepath;

        printData(); // Выводим данные в консоль
    }
}
```

```

initComponents();
getContentPane().setBackground(new java.awt.Color(225, 249, 247));
getContentPane().setBackground(new java.awt.Color(225, 249, 247));

JTableHeader header1 = jTable1.getTableHeader();
header1.setBackground(new Color(161, 227, 238));
header1.setForeground(new Color(13, 14, 57));
header1.setFont(new Font("Bahnschrift", Font.PLAIN, 12));

JTableHeader header2 = jTable2.getTableHeader();
header2.setBackground(new Color(161, 227, 238));
header2.setForeground(new Color(13, 14, 57));
header2.setFont(new Font("Bahnschrift", Font.PLAIN, 12));

JTableHeader header3 = jTable3.getTableHeader();
header3.setBackground(new Color(161, 227, 238));
header3.setForeground(new Color(13, 14, 57));
header3.setFont(new Font("Bahnschrift", Font.PLAIN, 12));

fillTables(); // Заполняем таблицы
}

public Edit() {
    this.rules = null;
    this.conditions = null;
    this.asks = new ArrayList<>();

    System.out.println("parametrov net");
    initComponents();
}

private void printData() {
    System.out.println("\nFile: " + (filename != null ? filename : "No
information"));
    System.out.println("Path: " + (filepath != null ? filepath : "No
information"));

    System.out.println("Transmitted rules:");
    for (Rule rule : rules) {
        System.out.println(rule);
    }

    System.out.println("\nTransmitted conditions:");

```

```

        for (Condition condition : conditions) {
            System.out.println(condition);
        }

        System.out.println("\nTransmitted asks:");
        for (Ask ask : asks) {
            System.out.println(ask);
        }
    }

    public void updateRulesTable() {
        DefaultTableModel model = (DefaultTableModel) jTable1.getModel();
        model.setRowCount(0); // Очищаем таблицу

        for (Rule rule : rules) {
            model.addRow(new Object[]{rule.getName()});
        }
    }

    public void updateConditionsTable() {
        DefaultTableModel model = (DefaultTableModel) jTable2.getModel();
        model.setRowCount(0); // Очистка таблицы перед обновлением

        for (Condition condition : conditions) {
            model.addRow(new Object[]{condition.getName()});
        }
    }

    public void updateAsksTable() {

        DefaultTableModel modelAsks = (DefaultTableModel) jTable3.getModel();
        modelAsks.setRowCount(0); // Очищаем таблицу перед обновлением

        for (Ask ask : asks) {
            for (int conditionId : ask.getConditionIds()) {
                Condition condition = findConditionById(conditionId);
                if (condition != null && condition.getName() != null &&
!condition.getName().trim().isEmpty()) {
                    modelAsks.addRow(new          Object[]{ask.getQuestion(),
condition.getName()});
                }
            }
        }

        //for (Condition condition : conditions) {
            //String questionText = ""; // По умолчанию пусто

```

```

        // Ищем соответствующий вопрос по ID утверждения
        //for (Ask ask : asks) {
            //if (ask.getId() == condition.getId()) {
                //questionText = ask.getQuestion();
                //break;
            //}
        //}

        // Добавляем строку в таблицу (если утверждение есть, но вопроса
нет, то оставляем пустым)
        //modelAsks.addRow(new                                     Object[]{questionText,
condition.getName()});
        //}

    }

    // Метод для поиска названия утверждения по ID
    //private String getConditionNameById(int conditionId) {
    //    for (Condition condition : conditions) {
    //        if (condition.getId() == conditionId) {
    //            return condition.getName();
    //        }
    //    }
    //    return "Неизвестное утверждение"; // На случай ошибки
    //}

    private Condition findConditionById(int id) {
        for (Condition condition : conditions) {
            if (condition.getId() == id) {
                return condition;
            }
        }
        return null; // Если утверждение не найдено
    }

    private void fillTables() {
        DefaultTableModel modelRules = (DefaultTableModel)
jTable1.getModel();
        DefaultTableModel modelConditions = (DefaultTableModel)
jTable2.getModel();
        DefaultTableModel modelAsks = (DefaultTableModel) jTable3.getModel();

        // Очищаем таблицы перед заполнением
        modelRules.setRowCount(0);
        modelConditions.setRowCount(0);
        modelAsks.setRowCount(0); // Очищаем таблицу перед обновлением
    }

```

```

        // Заполняем таблицу правил
        for (Rule rule : rules) {
            modelRules.addRow(new Object[]{rule.getName()}); // Добавляем
строку
        }

        // Заполняем таблицу утверждений
        for (Condition condition : conditions) {
            modelConditions.addRow(new Object[]{condition.getName()});
        }

        // Создаем отображаемый список вопросов и утверждений
        for (Ask ask : asks) {
            String questionText = ask.getQuestion(); // Берем сам вопрос

            // Перебираем все ID условий, связанных с этим вопросом
            for (int conditionId : ask.getConditionIds()) {
                Condition condition = findConditionById(conditionId);

                // Проверяем, что и вопрос, и утверждение существуют
                if (questionText != null && !questionText.trim().isEmpty() &&
                    condition != null && condition.getName() != null &&
!condition.getName().trim().isEmpty()) {

                    // Добавляем строку в таблицу
                    modelAsks.addRow(new Object[]{questionText,
condition.getName()});
                }
            }
        }

        private void saveRulesToFile() {
            if (filepath == null || filename == null) {
                // Если файл не выбран, предложить сохранить в новый
                JFileChooser fileChooser = new JFileChooser();
                fileChooser.setDialogTitle("Выберите место для сохранения базы
знаний");

                int userSelection = fileChooser.showSaveDialog(this);

                if (userSelection == JFileChooser.APPROVE_OPTION) {
                    File newFile = fileChooser.getSelectedFile();
                    filename = newFile.getName();
                }
            }
        }
    }
}

```

```

        filepath = newFile.getAbsolutePath();

        try {
            if (newFile.createNewFile()) {
                JOptionPane.showMessageDialog(this, "Файл создан: " +
newFile.getAbsolutePath());
            }
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Ошибка при создании
файла!", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }
    } else {
        return; // Если пользователь отменил, ничего не делаем
    }
}

// Теперь записываем правила в файл
try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filepath))) {
    for (Rule rule : rules) {
        // Формируем строку в нужном формате
        String ruleString = String.format(
            "rule(%d; %s; %s; %s)",
            rule.getId(),
            rule.getCategory(),
            rule.getName(),
            rule.getConditionIds().toString()
        );

        writer.write(ruleString);
        writer.newLine();
    }

    writer.newLine();

    // Записываем утверждения (Conditions)
    for (Condition condition : conditions) {
        String conditionString = String.format(
            "cond(%d; %s)",
            condition.getId(),
            condition.getName()
        );

        writer.write(conditionString);
        writer.newLine();
    }
}

```

```

        writer.newLine();

        for (Ask ask : asks) {
            String conditionIdsString =
ask.getConditionIds().toString().replace(" ", ""); // Убираем пробелы
            String askString = String.format(
                "ask(%d; %s; %s)",
                ask.getId(),
                ask.getQuestion(),
                conditionIdsString
            );
            writer.write(askString);
            writer.newLine();
        }

        writer.newLine();

        // Записываем информацию о файле
        String knowledgeBaseInfo = String.format("knowledge_base_file:
%s", filename);
        writer.write(knowledgeBaseInfo);
        writer.newLine();

        JOptionPane.showMessageDialog(this, "База знаний успешно
сохранена!", "Успех", JOptionPane.INFORMATION_MESSAGE);
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "Ошибка при сохранении
файла!", "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}

/**
 * This method is called from within the constructor to initialize the
form.
 *
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">

```



```

private void initComponents() {

    jScrollPane1 = new javax.swing.JScrollPane();
    jTable1 = new javax.swing.JTable();
    jScrollPane2 = new javax.swing.JScrollPane();
    jTable2 = new javax.swing.JTable();
    jButton1 = new javax.swing.JButton();
    jButton2 = new javax.swing.JButton();
    jButton3 = new javax.swing.JButton();
    jButton4 = new javax.swing.JButton();
    jButton5 = new javax.swing.JButton();
    jButton6 = new javax.swing.JButton();
    jButton7 = new javax.swing.JButton();
    jButton8 = new javax.swing.JButton();
    jScrollPane3 = new javax.swing.JScrollPane();
    jTable3 = new javax.swing.JTable();
    jButton9 = new javax.swing.JButton();
    jButton10 = new javax.swing.JButton();
    jButton11 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

    jTable1.setBackground(new java.awt.Color(184, 240, 236));
    jTable1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jTable1.setForeground(new java.awt.Color(13, 14, 57));
    jTable1.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {

            },
        new String [] {
            "Объекты"
        }
    ) {
        Class[] types = new Class [] {
            java.lang.String.class
        };
        boolean[] canEdit = new boolean [] {
            false
        };

        public Class getColumnClass(int columnIndex) {
            return types [columnIndex];
        }

        public boolean isCellEditable(int rowIndex, int columnIndex) {

```

```

        return canEdit [columnIndex];
    }
});
jScrollPane1.setViewportViewView(jTable1);
if (jTable1.getColumnModel().getColumnCount() > 0) {
    jTable1.getColumnModel().getColumn(0).setResizable(false);
}

jTable2.setBackground(new java.awt.Color(184, 240, 236));
jTable2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jTable2.setForeground(new java.awt.Color(13, 14, 57));
jTable2.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

        },
        new String [] {
            "Утверждения"
        }
    ) {
        Class[] types = new Class [] {
            java.lang.String.class
        };
        boolean[] canEdit = new boolean [] {
            false
        };

        public Class getColumnClass(int columnIndex) {
            return types [columnIndex];
        }

        public boolean isCellEditable(int rowIndex, int columnIndex) {
            return canEdit [columnIndex];
        }
    });
jScrollPane2.setViewportViewView(jTable2);
if (jTable2.getColumnModel().getColumnCount() > 0) {
    jTable2.getColumnModel().getColumn(0).setResizable(false);
}

jButton1.setBackground(new java.awt.Color(125, 202, 233));
jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton1.setForeground(new java.awt.Color(13, 14, 57));
jButton1.setText("Добавить");
jButton1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {

```

```

        jButton1ActionPerformed(evt);
    }
});

jButton2.setBackground(new java.awt.Color(125, 202, 233));
jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton2.setForeground(new java.awt.Color(13, 14, 57));
jButton2.setText("ИЗМЕНИТЬ");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});

jButton3.setBackground(new java.awt.Color(125, 202, 233));
jButton3.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton3.setForeground(new java.awt.Color(13, 14, 57));
jButton3.setText("УДАЛИТЬ");
jButton3.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton3ActionPerformed(evt);
    }
});

jButton4.setBackground(new java.awt.Color(125, 202, 233));
jButton4.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton4.setForeground(new java.awt.Color(13, 14, 57));
jButton4.setText("ДОБАВИТЬ");
jButton4.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton4ActionPerformed(evt);
    }
});

jButton5.setBackground(new java.awt.Color(125, 202, 233));
jButton5.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton5.setForeground(new java.awt.Color(13, 14, 57));
jButton5.setText("ИЗМЕНИТЬ");
jButton5.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton5ActionPerformed(evt);
    }
});

jButton6.setBackground(new java.awt.Color(125, 202, 233));

```

```

jButton6.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton6.setForeground(new java.awt.Color(13, 14, 57));
jButton6.setText("Удалить");
jButton6.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton6ActionPerformed(evt);
    }
});

jButton7.setBackground(new java.awt.Color(125, 202, 233));
jButton7.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton7.setForeground(new java.awt.Color(13, 14, 57));
jButton7.setText("Назад");
jButton7.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton7ActionPerformed(evt);
    }
});

jButton8.setBackground(new java.awt.Color(125, 202, 233));
jButton8.setText("Сохранить");
jButton8.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton8ActionPerformed(evt);
    }
});

jTable3.setBackground(new java.awt.Color(184, 240, 236));
jTable3.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jTable3.setForeground(new java.awt.Color(13, 14, 57));
jTable3.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

        },
    new String [] {
        "Вопрос", "Утверждение"
    }
) {
    Class[] types = new Class [] {
        java.lang.String.class, java.lang.String.class
    };
    boolean[] canEdit = new boolean [] {
        false, false
    };
};

```

```

        public Class getColumnClass(int columnIndex) {
            return types [columnIndex];
        }

        public boolean isCellEditable(int rowIndex, int columnIndex) {
            return canEdit [columnIndex];
        }
    });
    jScrollPane3.setViewportViewView(jTable3);
    if (jTable3.getColumnModel().getColumnCount() > 0) {
        jTable3.getColumnModel().getColumn(0).setResizable(false);
        jTable3.getColumnModel().getColumn(1).setResizable(false);
    }

    jButton9.setBackground(new java.awt.Color(125, 202, 233));
    jButton9.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton9.setForeground(new java.awt.Color(13, 14, 57));
    jButton9.setText("Добавить");
    jButton9.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton9ActionPerformed(evt);
        }
    });

    jButton10.setBackground(new java.awt.Color(125, 202, 233));
    jButton10.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton10.setForeground(new java.awt.Color(13, 14, 57));
    jButton10.setText("Изменить");
    jButton10.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton10ActionPerformed(evt);
        }
    });

    jButton11.setBackground(new java.awt.Color(125, 202, 233));
    jButton11.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton11.setForeground(new java.awt.Color(13, 14, 57));
    jButton11.setText("Удалить");
    jButton11.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton11ActionPerformed(evt);
        }
    });

```

```

        javax.swing.GroupLayout layout = new
        javax.swing.GroupLayout(getContentPane());

```

```

        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup())

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,                205,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addGroup(layout.createSequentialGroup()
                    .addGap(7, 7, 7)
                    .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 95, javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE,                95,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addGroup(layout.createSequentialGroup()
                    .addContainerGap()
                    .addComponent(jButton3,
javax.swing.GroupLayout.PREFERRED_SIZE,                95,
javax.swing.GroupLayout.PREFERRED_SIZE))
                    .addGroup(layout.createSequentialGroup()
                        .addContainerGap()
                        .addComponent(jButton7,
javax.swing.GroupLayout.PREFERRED_SIZE,                95,
javax.swing.GroupLayout.PREFERRED_SIZE)))

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup())

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(jButton8,
javax.swing.GroupLayout.PREFERRED_SIZE,                95,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addGroup(layout.createSequentialGroup()
                    .addGap(12, 12, 12)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jScrollPane2,
javax.swing.GroupLayout.PREFERRED_SIZE, 0, Short.MAX_VALUE)
                .addGroup(layout.createSequentialGroup())

```

```

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addComponent(jButton4,
javax.swing.GroupLayout.PREFERRED_SIZE, 95, javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jButton5,
javax.swing.GroupLayout.PREFERRED_SIZE, 95,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addComponent(jButton6,
javax.swing.GroupLayout.PREFERRED_SIZE, 95,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addGap(0, 31, Short.MAX_VALUE)))
        .addGap(18, 18, 18)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addGap(0, 0, Short.MAX_VALUE)
                .addComponent(jScrollPane3,
javax.swing.GroupLayout.PREFERRED_SIZE, 648,
javax.swing.GroupLayout.PREFERRED_SIZE))
            .addGroup(layout.createSequentialGroup()

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING,
false)
                .addComponent(jButton11,
javax.swing.GroupLayout.Alignment.LEADING, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(jButton9,
javax.swing.GroupLayout.Alignment.LEADING, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                .addComponent(jButton10)
                .addGap(0, 0, Short.MAX_VALUE))))
        .addContainerGap()
    );
    layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(41, 41, 41)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jScrollPane2,
javax.swing.GroupLayout.PREFERRED_SIZE, 272,
javax.swing.GroupLayout.PREFERRED_SIZE)

```

```

        .addComponent(jScrollPane3,
javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGroup(layout.createSequentialGroup()
        .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jButton1)
        .addComponent(jButton2)
        .addComponent(jButton4)
        .addComponent(jButton5)
        .addComponent(jButton9)
        .addComponent(jButton10)))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(jButton3)
        .addComponent(jButton6)
        .addComponent(jButton11))
        .addGap(53, 53, 53)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jButton7)
        .addComponent(jButton8))
        .addContainerGap(18, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // Добавить объект
    AddObject addObjectWindow = new AddObject(this, rules, conditions);
    addObjectWindow.setVisible(true);
}

private void jButton8ActionPerformed(java.awt.event.ActionEvent evt) {
    // сохраняем изменение в файле
    saveRulesToFile();
}

private void jButton7ActionPerformed(java.awt.event.ActionEvent evt) {

```



```

        // TODO add your handling code here:
        this.dispose();
    }

    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
        // редактировать объект
        // Получаем выбранную строку из таблицы
        int selectedRow = jTable1.getSelectedRow();
        System.out.println("Selected Row: " + selectedRow);

        // Проверяем, что строка выбрана
        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Выберите элемент для редактирования", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Получаем название правила из выбранной строки таблицы
        String selectedRuleName = (String) jTable1.getValueAt(selectedRow, 0);
        // 0 - колонка с названием

        // Находим объект Rule по названию
        Rule selectedRule = rules.stream()
            .filter(rule -> rule.getName().equals(selectedRuleName))
            .findFirst()
            .orElse(null);

        if (selectedRule == null) {
            JOptionPane.showMessageDialog(this, "Ошибка: не удалось найти объект!", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        int objectId = selectedRule.getId(); // Теперь ID определен корректно

        System.out.println("Selected Row: " + selectedRow);
        System.out.println("Object ID: " + objectId);

        // Ищем объект с этим id в списке правил
        Rule ruleToEdit = rules.stream()
            .filter(rule -> rule.getId() == objectId)
            .findFirst()
            .orElse(null);

        // Если объект найден, открываем окно редактирования
    }

```

```

        if (ruleToEdit != null) {
            // Передаем найденный объект в EditObject для редактирования
            EditObject editObject = new EditObject(this, rules, conditions,
ruleToEdit);

            editObject.setVisible(true); // Показываем окно редактирования
        } else {
            // Если объект с таким id не найден, выводим ошибку
            JOptionPane.showMessageDialog(this, "Объект с таким id не найден",
"Ошибка", JOptionPane.ERROR_MESSAGE);
        }
    }

    private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {
        /// Получаем выбранную строку
        int selectedRow = jTable1.getSelectedRow();

        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Выберите элемент для
удаления", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Получаем имя объекта из таблицы
        String ruleName = (String) jTable1.getValueAt(selectedRow, 0);

        // Ищем объект Rule с этим именем
        Rule ruleToRemove = null;
        for (Rule rule : rules) {
            if (rule.getName().equals(ruleName)) {
                ruleToRemove = rule;
                break;
            }
        }

        if (ruleToRemove != null) {
            // Подтверждение удаления
            int confirm = JOptionPane.showConfirmDialog(this,
"Вы уверены, что хотите удалить " + ruleToRemove.getName() +
"?",
"Подтверждение удаления", JOptionPane.YES_NO_OPTION);

            if (confirm == JOptionPane.YES_OPTION) {
                rules.remove(ruleToRemove); // Удаляем из списка
                updateRulesTable(); // Обновляем таблицу
            }
        } else {

```

```

        JOptionPane.showMessageDialog(this, "Ошибка: Объект не найден",
"Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}

private void jButton6ActionPerformed(java.awt.event.ActionEvent evt) {
    // Получаем выбранную строку в таблице утверждений (jTable2)
    int selectedRow = jTable2.getSelectedRow();

    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Выберите утверждение для
удаления", "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Получаем имя утверждения из таблицы
    String conditionName = (String) jTable2.getValueAt(selectedRow, 0);

    // Ищем объект Condition с этим именем
    Condition conditionToRemove = null;
    for (Condition condition : conditions) {
        if (condition.getName().equals(conditionName)) {
            conditionToRemove = condition;
            break;
        }
    }

    if (conditionToRemove != null) {
        // Подтверждение удаления
        int confirm = JOptionPane.showConfirmDialog(this,
            "Вы уверены, что хотите удалить утверждение: " +
conditionToRemove.getName() + "?",
            "Подтверждение удаления", JOptionPane.YES_NO_OPTION);

        if (confirm == JOptionPane.YES_OPTION) {
            int conditionIdToRemove = conditionToRemove.getId(); // ID
удаляемого утверждения
            conditions.remove(conditionToRemove); // Удаляем из списка
            updateConditionsTable(); // Обновляем таблицу утверждений

            // Удаляем соответствующий вопрос (Ask) из списка
            Ask askToRemove = null;
            for (Ask ask : asks) {
                if (ask.getId() == conditionIdToRemove) {
                    askToRemove = ask;
                    break;
                }
            }
        }
    }
}

```

```

        }
    }

    if (askToRemove != null) {
        asks.remove(askToRemove); // Удаляем из списка вопросов
    }

    updateAsksTable(); // Обновляем таблицу вопросов

    // Удаляем ID утверждения из всех правил
    for (Rule rule : rules) {
        List<Integer> updatedConditionIds = new
ArrayList<>(rule.getConditionIds());
        updatedConditionIds.removeIf(id -> id ==
conditionIdToRemove); // Удаляем ID
        rule.setConditionIds(updatedConditionIds); // Обновляем
rule
    }

    // Обновляем таблицу правил
    updateRulesTable();
}
} else {
    JOptionPane.showMessageDialog(this, "Ошибка: Утверждение не
найден", "Ошибка", JOptionPane.ERROR_MESSAGE);
}
}

private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {
    int selectedRow = jTable2.getSelectedRow();

    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Выберите утверждение для
редактирования", "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Получаем название утверждения из таблицы
    String selectedConditionName = (String)
jTable2.getValueAt(selectedRow, 0); // 0 - колонка с названием

    // Находим объект Condition по названию
    Condition selectedCondition = conditions.stream()
        .filter(condition ->
condition.getName().equals(selectedConditionName))
        .findFirst()
        .orElse(null);

```

```

        if (selectedCondition == null) {
            JOptionPane.showMessageDialog(this, "Ошибка: не удалось найти
утверждение!", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        int conditionId = selectedCondition.getId(); // Получаем правильный
ID
        System.out.println("Condition ID: " + conditionId);

        // Открываем окно редактирования
        EditCond editCondWindow = new EditCond(this, conditions, conditionId);
        editCondWindow.setVisible(true);
    }

    private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
        EditCond editCondWindow = new EditCond(this, conditions, null); //
Передаем null для нового условия
        editCondWindow.setVisible(true);
    }

    private void jButton9ActionPerformed(java.awt.event.ActionEvent evt) {
        AddAsk addAskWindow = new AddAsk(this, asks, conditions);
        addAskWindow.setVisible(true);
    }

    private void jButton10ActionPerformed(java.awt.event.ActionEvent evt) {
        // Получаем выделенную строку
        int selectedRow = jTable3.getSelectedRow();

        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Выберите вопрос для
редактирования!", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Получаем текст вопроса из таблицы
        String questionText = (String) jTable3.getValueAt(selectedRow, 0);

        // Ищем вопрос в списке asks по тексту (можно по ID, если он хранится
в таблице)
        Ask selectedAsk = null;
        for (Ask ask : asks) {
            if (ask.getQuestion().equals(questionText)) {
                selectedAsk = ask;
            }
        }
    }

```

```

        break;
    }
}

if (selectedAsk == null) {
    JOptionPane.showMessageDialog(this, "Ошибка: Вопрос не найден!",
    "Ошибка", JOptionPane.ERROR_MESSAGE);
    return;
}

// Создаем список утверждений, которые можно выбрать (только те, у
которых нет вопросов, + текущее)
List<Condition> availableConditions = new ArrayList<>();
for (Condition condition : conditions) {
    boolean hasAsk = false;
    for (Ask ask : asks) {
        if (ask.getConditionIds().contains(condition.getId())    &&
ask.getId() != selectedAsk.getId()) {
            hasAsk = true;
            break;
        }
    }
    if (!hasAsk    ||
selectedAsk.getConditionIds().contains(condition.getId())) {
        availableConditions.add(condition);
    }
}

// Открываем окно редактирования с выбранным вопросом
EditAsk editAskWindow = new EditAsk(this, asks, availableConditions,
selectedAsk.getId());
editAskWindow.setVisible(true);
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // Получаем выбранную строку в таблице вопросов
    int selectedRow = jTable3.getSelectedRow();

    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Выберите вопрос для
удаления", "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Получаем текст вопроса и утверждения из таблицы
    String questionText = (String) jTable3.getValueAt(selectedRow, 0);
    String conditionText = (String) jTable3.getValueAt(selectedRow, 1);

```

```

// Находим соответствующий объект Ask
Ask askToRemove = null;
for (Ask ask : asks) {
    if (ask.getQuestion().equals(questionText)) {
        askToRemove = ask;
        break;
    }
}

if (askToRemove != null) {
    // Подтверждение удаления
    int confirm = JOptionPane.showConfirmDialog(this,
        "Вы уверены, что хотите удалить вопрос: \"" +
askToRemove.getQuestion() + "\"?",
        "Подтверждение удаления", JOptionPane.YES_NO_OPTION);

    if (confirm == JOptionPane.YES_OPTION) {
        asks.remove(askToRemove); // Удаляем из списка
        updateAsksTable(); // Обновляем таблицу вопросов
    }
} else {
    JOptionPane.showMessageDialog(this, "Ошибка: Вопрос не найден",
"Ошибка", JOptionPane.ERROR_MESSAGE);
}
}
/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
        *
        * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
        */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    }
}

```

```

        } catch (ClassNotFoundException ex) {

java.util.logging.Logger.getLogger(Edit.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);

        } catch (InstantiationException ex) {

java.util.logging.Logger.getLogger(Edit.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);

        } catch (IllegalAccessException ex) {

java.util.logging.Logger.getLogger(Edit.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);

        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(Edit.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);

        }
    }
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        //new Edit().setVisible(true);
    }
});
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton10;
private javax.swing.JButton jButton11;
private javax.swing.JButton jButton2;
private javax.swing.JButton jButton3;
private javax.swing.JButton jButton4;
private javax.swing.JButton jButton5;
private javax.swing.JButton jButton6;
private javax.swing.JButton jButton7;
private javax.swing.JButton jButton8;
private javax.swing.JButton jButton9;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JScrollPane jScrollPane2;
private javax.swing.JScrollPane jScrollPane3;
private javax.swing.JTable jTable1;
private javax.swing.JTable jTable2;
private javax.swing.JTable jTable3;
// End of variables declaration
}

```


Приложение А.7. EditAsk.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GuiForms/JFrame.java   to
edit this template
 */
package pskslabal;

import javax.swing.*.*;
import javax.swing.table.DefaultTableModel;
import java.util.List;

/**
 *
 * @author Вика
 */
public class EditAsk extends javax.swing.JFrame {
    private Edit parent;
    private List<Condition> conditions;
    private List<Ask> asks;
    private Ask AskToEdit; // Редактируемый вопрос

    /**
     * Creates new form EditAsk
     */
    public EditAsk(Edit parent, List<Ask> asks, List<Condition> conditions,
int askId) {
        this.parent = parent;
        this.asks = asks;
        this.conditions = conditions;

        // Находим редактируемый вопрос
        for (Ask ask : asks) {
            if (ask.getId() == askId) {
                this.AskToEdit = ask;
                break;
            }
        }

        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
        fillTable(); // Заполняем таблицу
        loadAskData(); // Загружаем данные вопроса
    }
}
```

```

public EditAsk() {
    initComponents();
}

private void fillTable() {
    DefaultTableModel model = (DefaultTableModel) jTable1.getModel();
    model.setRowCount(0); // Очищаем таблицу

    for (Condition condition : conditions) {
        boolean isSelected =
AskToEdit.getConditionIds().contains(condition.getId());
        model.addRow(new Object[]{condition.getName(), isSelected});
    }

    // Добавляем слушатель событий для чекбоксов
    jTable1.getModel().addTableModelListener(e -> {
        int selectedCount = 0;

        for (int i = 0; i < model.getRowCount(); i++) {
            Boolean isChecked = (Boolean) model.getValueAt(i, 1);
            if (isChecked != null && isChecked) {
                selectedCount++;
                if (selectedCount > 1) {
                    JOptionPane.showMessageDialog(this, "Можно выбрать
только одно утверждение!", "Ошибка", JOptionPane.ERROR_MESSAGE);
                    model.setValueAt(false, i, 1); // Сбрасываем последний
выбранный чекбокс

                    break;
                }
            }
        }
    });
}

private void loadAskData() {
    if (AskToEdit != null) {
        jTextField1.setText(AskToEdit.getQuestion());
    }
}

/**
 * This method is called from within the constructor to initialize the
form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.

```

```

    */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        jLabel1 = new javax.swing.JLabel();
        jTextField1 = new javax.swing.JTextField();
        jScrollPane1 = new javax.swing.JScrollPane();
        jTable1 = new javax.swing.JTable();
        jButton1 = new javax.swing.JButton();
        jButton2 = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

        jLabel1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
        jLabel1.setForeground(new java.awt.Color(13, 14, 57));
        jLabel1.setText("Вопрос");

        jTextField1.setBackground(new java.awt.Color(184, 240, 236));
        jTextField1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); //
NOI18N
        jTextField1.setForeground(new java.awt.Color(13, 14, 57));

        jTable1.setBackground(new java.awt.Color(184, 240, 236));
        jTable1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
        jTable1.setForeground(new java.awt.Color(13, 14, 57));
        jTable1.setModel(new javax.swing.table.DefaultTableModel(
            new Object [][] {

                },
            new String [] {
                "Утверждение", "Относится"
            }
        ) {
            Class[] types = new Class [] {
                java.lang.String.class, java.lang.Boolean.class
            };
            boolean[] canEdit = new boolean [] {
                false, true
            };

            public Class getColumnClass(int columnIndex) {
                return types [columnIndex];
            }
        }
    }

```

```

        public boolean isCellEditable(int rowIndex, int columnIndex) {
            return canEdit [columnIndex];
        }
    });

jTable1.setSelectionMode(javax.swing.ListSelectionModel.SINGLE_SELECTION);
jScrollPane1.setViewportViewView(jTable1);
if (jTable1.getColumnModel().getColumnCount() > 0) {
    jTable1.getColumnModel().getColumn(0).setResizable(false);
    jTable1.getColumnModel().getColumn(1).setResizable(false);
}

jButton1.setBackground(new java.awt.Color(125, 202, 233));
jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton1.setForeground(new java.awt.Color(13, 14, 57));
jButton1.setText("Добавить");
jButton1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton1ActionPerformed(evt);
    }
});

jButton2.setBackground(new java.awt.Color(125, 202, 233));
jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton2.setForeground(new java.awt.Color(13, 14, 57));
jButton2.setText("Назад");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
        .addContainerGap(38, Short.MAX_VALUE)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(138, 138, 138)
        .addComponent(jLabel1))

```

```

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING,
false)

        .addGroup(layout.createSequentialGroup()

            .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 89, javax.swing.GroupLayout.PREFERRED_SIZE)

            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

            .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE,                                89,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addComponent(jScrollPane1,
javax.swing.GroupLayout.Alignment.LEADING, javax.swing.GroupLayout.PREFERRED_SIZE,
0, Short.MAX_VALUE)

            .addComponent(jTextField1,
javax.swing.GroupLayout.Alignment.LEADING, javax.swing.GroupLayout.PREFERRED_SIZE,
324, javax.swing.GroupLayout.PREFERRED_SIZE))

            .addGap(37, 37, 37))

        );

layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addGroup(layout.createSequentialGroup()

            .addGap(15, 15, 15)

            .addComponent(jTextField1,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

            .addComponent(jLabel1)

            .addGap(18, 18, 18)

            .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,                                155,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addGap(18, 18, 18)

            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

                .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 33, javax.swing.GroupLayout.PREFERRED_SIZE)

                .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE,                                33,
javax.swing.GroupLayout.PREFERRED_SIZE)

                .addContainerGap(17, Short.MAX_VALUE))

            );

        pack();
} // </editor-fold>

```

```

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    String updatedQuestion = jTextField1.getText().trim();

    if (updatedQuestion.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Введите текст вопроса!",
"Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Получаем список выбранных утверждений (только одно!)
    List<Integer> selectedConditionIds = AskToEdit.getConditionIds();
    selectedConditionIds.clear();

    DefaultTableModel model = (DefaultTableModel) jTable1.getModel();
    for (int i = 0; i < model.getRowCount(); i++) {
        Boolean isChecked = (Boolean) model.getValueAt(i, 1);
        if (isChecked != null && isChecked) {
            for (Condition condition : conditions) {
                if (condition.getName().equals(model.getValueAt(i, 0))) {
                    selectedConditionIds.add(condition.getId());
                    break;
                }
            }
        }
    }

    if (selectedConditionIds.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Выберите одно утверждение!",
"Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Обновляем данные вопроса
    AskToEdit.setQuestion(updatedQuestion);
    AskToEdit.setConditionIds(selectedConditionIds);

    // Обновляем таблицу в Edit
    parent.updateAsksTable();

    // Закрываем окно
    this.dispose();
}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    this.dispose();
}

```

```

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
        *
        * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
        */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(EditAsk.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(EditAsk.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(EditAsk.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(EditAsk.class.getName()).log(java.util.logging.
Level.SEVERE, null, ex);
    }
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new EditAsk().setVisible(true);
    }
});
}

```

```
// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JLabel jLabel1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable jTable1;
private javax.swing.JTextField jTextField1;
// End of variables declaration
}
```


Приложение А.8. EditCond.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GuiForms/JFrame.java   to
edit this template
 */
package pskslabal;

import javax.swing.*.*;
import java.util.List;
import java.util.Optional;

/**
 *
 * @author Вика
 */
public class EditCond extends javax.swing.JFrame {

    private Edit parent;
    private List<Condition> conditions;
    private Integer conditionId; // ID утверждения (null, если создаем новое)

    /**
     * Creates new form EditCond
     */

    public EditCond(Edit parent, List<Condition> conditions, Integer
conditionId) {
        this.parent = parent;
        this.conditions = conditions;
        this.conditionId = conditionId;

        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
        setupForm(); // Заполняем текстовое поле
    }

    public EditCond() {
        initComponents();
    }

    private void setupForm() {
        if (conditionId != null) { // Если редактируем
            Optional<Condition> condOpt = conditions.stream()
```

```

        .filter(cond -> cond.getId() == (conditionId))
        .findFirst();

        condOpt.ifPresent(cond -> jTextField1.setText(cond.getName()));
    } else {
        jTextField1.setText(""); // Если добавляем новое утверждение -
очищаем поле
    }
}

/**
 * This method is called from within the constructor to initialize the
form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jTextField1 = new javax.swing.JTextField();
    jLabel1 = new javax.swing.JLabel();
    jButton1 = new javax.swing.JButton();
    jButton2 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

    jTextField1.setBackground(new java.awt.Color(184, 240, 236));
    jTextField1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); //
NOI18N
    jTextField1.setForeground(new java.awt.Color(13, 14, 57));

    jLabel1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jLabel1.setForeground(new java.awt.Color(13, 14, 57));
    jLabel1.setText("Название утверждения");

    jButton1.setBackground(new java.awt.Color(125, 202, 233));
    jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton1.setForeground(new java.awt.Color(13, 14, 57));
    jButton1.setText("Назад");
    jButton1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton1ActionPerformed(evt);
        }
    });
}

```

```

jButton2.setBackground(new java.awt.Color(125, 202, 233));
jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton2.setForeground(new java.awt.Color(13, 14, 57));
jButton2.setText("Сохранить");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(132, 132, 132)
        .addComponent(jLabel1)
        .addGroup(layout.createSequentialGroup()
            .addGap(34, 34, 34)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
    .addComponent(jTextField1,
javax.swing.GroupLayout.PREFERRED_SIZE, 321,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
    .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 100,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 100,
javax.swing.GroupLayout.PREFERRED_SIZE)))
    .addGap(44, Short.MAX_VALUE))
);
layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

```

```

        .addGroup(layout.createSequentialGroup())
            .addGap(30, 30, 30)
            .addComponent(jTextField1,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(jLabel1)
            .addGap(27, 27, 27)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 37, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 37,
javax.swing.GroupLayout.PREFERRED_SIZE))
            .addContainerGap(20, Short.MAX_VALUE))
    );

    pack();
} // </editor-fold>

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    String conditionName = jTextField1.getText().trim();
    if (conditionName.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Введите название
утверждения!", "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    if (conditionId != null) {
        // Изменяем существующее утверждение
        conditions.stream()
            .filter(cond -> cond.getId() == (conditionId))
            .findFirst()
            .ifPresent(cond -> cond.setName(conditionName));
    } else {
        // Получаем максимальный ID + 1
        int newId = conditions.stream()
            .mapToInt(Condition::getId)
            .max()
            .orElse(0) + 1;

        conditions.add(new Condition(newId, conditionName));
    }
}

```

```

        // Обновляем таблицу
        parent.updateConditionsTable();
        this.dispose();
    }

    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        this.dispose();
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
           *
           * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
           */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {
            java.util.logging.Logger.getLogger(EditCond.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);
        } catch (InstantiationException ex) {
            java.util.logging.Logger.getLogger(EditCond.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);
        } catch (IllegalAccessException ex) {
            java.util.logging.Logger.getLogger(EditCond.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(EditCond.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);
        }
    }
}
//</editor-fold>

```

```

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new EditCond().setVisible(true);
            }
        });
    }

    // Variables declaration - do not modify
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration
}

```

Приложение А.9. EditObject.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GuiForms/JFrame.java   to
edit this template
 */
package pskslabal;

import java.util.List;
import java.util.ArrayList;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import pskslabal.Rule; // Для использования класса Rule
import pskslabal.Condition; // Для использования класса Condition

/**
 *
 * @author Вика
 */
public class EditObject extends javax.swing.JFrame {
    private Edit parent;
    private List<Rule> rules;
    private List<Condition> conditions;
    private Rule ruleToEdit; // Новый объект для редактирования

    /**
     * Creates new form EditObject
     */
    public EditObject(Edit parent, List<Rule> rules, List<Condition>
conditions, Rule ruleToEdit) {
        this.parent = parent;
        this.rules = rules;
        this.conditions = conditions;
        this.ruleToEdit = ruleToEdit;

        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
        fillForm();
    }

    // private Condition findConditionById(Integer id) {
    //     for (Condition condition : conditions) {
    //         if (condition.getId()== id) {
    //             return condition;
    //         }
    //     }
    // }
```

```

//      }
//      return null; // Если не нашли
//  }

private void fillForm() {
    // Устанавливаем имя объекта в jTextField1
    jTextField1.setText(ruleToEdit.getName());

    // Заполняем таблицу jTable1 утверждениями
    DefaultTableModel model = (DefaultTableModel) jTable1.getModel();

    // Очистить таблицу перед добавлением данных
    model.setRowCount(0);

    // Проверяем, не null ли ruleToEdit
    List<Integer> ruleConditionIds = (ruleToEdit != null) ?
ruleToEdit.getConditionIds() : new ArrayList<>();

    // Заполняем таблицу всеми условиями из списка conditions
    for (Condition condition : conditions) {
        boolean isSelected =
ruleConditionIds.contains(condition.getId());
        model.addRow(new Object[]{condition.getName(), isSelected});
    }
}

/**
 * This method is called from within the constructor to initialize the
form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jLabel1 = new javax.swing.JLabel();
    jTextField1 = new javax.swing.JTextField();
    jScrollPane1 = new javax.swing.JScrollPane();
    jTable1 = new javax.swing.JTable();
    jButton1 = new javax.swing.JButton();
    jButton2 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

```


NOI18N

```
jLabel1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jLabel1.setForeground(new java.awt.Color(13, 14, 57));
jLabel1.setText("Название");

jTextField1.setBackground(new java.awt.Color(184, 240, 236));
jTextField1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); //
jTextField1.setForeground(new java.awt.Color(13, 14, 57));
jTextField1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jTextField1ActionPerformed(evt);
    }
});

jTable1.setBackground(new java.awt.Color(184, 240, 236));
jTable1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jTable1.setForeground(new java.awt.Color(13, 14, 57));
jTable1.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

        },
    new String [] {
        "Утверждение", "Относится"
    }
) {
    Class[] types = new Class [] {
        java.lang.String.class, java.lang.Boolean.class
    };
    boolean[] canEdit = new boolean [] {
        false, true
    };

    public Class getColumnClass(int columnIndex) {
        return types [columnIndex];
    }

    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
});
jScrollPane1.setViewportView(jTable1);
if (jTable1.getColumnModel().getColumnCount() > 0) {
    jTable1.getColumnModel().getColumn(0).setResizable(false);
    jTable1.getColumnModel().getColumn(1).setResizable(false);
}
```

```

jButton1.setBackground(new java.awt.Color(125, 202, 233));
jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton1.setForeground(new java.awt.Color(13, 14, 57));
jButton1.setText("Добавить");
jButton1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton1ActionPerformed(evt);
    }
});

jButton2.setBackground(new java.awt.Color(125, 202, 233));
jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton2.setForeground(new java.awt.Color(13, 14, 57));
jButton2.setText("Отмена");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup())
        .addGap(40, 40, 40)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup())
        .addGap(129, 129, 129)
        .addComponent(jLabel1))

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)

.addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
    .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 102,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 102,
javax.swing.GroupLayout.PREFERRED_SIZE))

```

```

        .addComponent(jTextField1)
        .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,          313,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addContainerGap(47, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(15, 15, 15)
            .addComponent(jTextField1,
javax.swing.GroupLayout.PREFERRED_SIZE,          javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(jLabel1)
            .addGap(18, 18, 18)
            .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,          158,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(18, 18, 18)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 32, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE,          32,
javax.swing.GroupLayout.PREFERRED_SIZE))
            .addContainerGap(17, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:

    String objectName = jTextField1.getText().trim();

    if (objectName.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Введите название объекта!",
"Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }
}

```

```

    }

    // Получаем список выбранных conditions (номера строк с галочкой)
    List<Integer> selectedConditionIds = new ArrayList<>();
    DefaultTableModel model = (DefaultTableModel) jTable1.getModel();

    for (int i = 0; i < model.getRowCount(); i++) {
        Boolean isChecked = (Boolean) model.getValueAt(i, 1); // Второй
столбец (чекбокс)
        if (isChecked != null && isChecked) {
            selectedConditionIds.add(conditions.get(i).getId()); // Берем
ID из списка conditions
        }
    }

    // Обновляем существующий объект Rule
    if (ruleToEdit != null) {
        ruleToEdit.setName(objectName);
        ruleToEdit.setConditionIds(selectedConditionIds);
    }
    // Вызываем метод обновления таблицы в Edit
    if (parent != null) {
        parent.updateRulesTable(); // !!! Вот тут исправленный вызов !!!
    }

    // Закрываем окно
    this.dispose();
}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    this.dispose();
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
        *
        * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
        */
    try {

```

```

        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {

javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {

java.util.logging.Logger.getLogger(EditObject.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);
        } catch (InstantiationException ex) {

java.util.logging.Logger.getLogger(EditObject.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);
        } catch (IllegalAccessException ex) {

java.util.logging.Logger.getLogger(EditObject.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(EditObject.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);
        }
    }
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {

    }

});
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JLabel jLabel1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable jTable1;
private javax.swing.JTextField jTextField1;
// End of variables declaration
}

```

Приложение А.10. FPSKSlab1.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GuiForms/JFrame.java   to
edit this template
 */
package pskslab1;

import javax.swing.JFileChooser;
import java.io.*;
import java.util.ArrayList;
import java.util.List;
import javax.swing.JOptionPane;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.ExecutionException;
import java.net.Socket;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import javax.swing.ImageIcon;
import javax.swing.JLabel;
import java.awt.Image;
import java.awt.Color;
import java.util.Comparator;
import javax.swing.JPanel;

/**
 *
 * @author Вика
 */
public class FPSKSlab1 extends javax.swing.JFrame {

    /**
     * Creates new form FPSKSlab1
     */
    private List<Rule> rules = new ArrayList<>();
    private List<Condition> conditions = new ArrayList<>();
    private List<Ask> asks = new ArrayList<>();
    private String filename = null;
    private String filepath = null;

    public FPSKSlab1() {
        initComponents();
    }
}
```

```

        setBackgroundImage();
        //getContentPane().setBackground(new java.awt.Color(225, 249, 247));
    }

    private void setBackgroundImage() {
        try {
            // 1. Загрузка изображения
            String imagePath = "D:\\Уник\\3 курс\\6
семестр\\PSKS\\курсовая\\PSKSlab1\\src\\image\\med.jpg";
            ImageIcon imageIcon = new ImageIcon(imagePath);

            // 2. Масштабирование изображения под размер окна
            Image image = imageIcon.getImage();
            Image scaledImage = image.getScaledInstance(
                getContentPane().getWidth(),
                getContentPane().getHeight(),
                Image.SCALE_SMOOTH
            );

            // 3. Создание фонового JLabel
            JLabel background = new JLabel(new ImageIcon(scaledImage));
            background.setBounds(0, 0, getWidth(), getHeight());

            // 4. Добавление на фоновый слой
            getLayeredPane().add(background,
Integer.valueOf(Integer.MIN_VALUE));

            // 5. Делаем contentPane прозрачным
            ((JPanel)getContentPane()).setOpaque(false);

        } catch (Exception e) {
            e.printStackTrace();
            // Если изображение не загрузится, используем цветной фон
            getContentPane().setBackground(new Color(225, 249, 247));
        }
    }

    /**
     * This method is called from within the constructor to initialize the
form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">

```

```

private void initComponents() {

    jButton2 = new javax.swing.JButton();
    jButton3 = new javax.swing.JButton();
    jButton1 = new javax.swing.JButton();
    jButton4 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
    setBackground(new java.awt.Color(40, 167, 215));

    jButton2.setBackground(new java.awt.Color(125, 202, 233));
    jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton2.setForeground(new java.awt.Color(13, 14, 57));
    jButton2.setText("Загрузить данные");
    jButton2.setToolTipText("");
    jButton2.setBorder(null);
    jButton2.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton2ActionPerformed(evt);
        }
    });

    jButton3.setBackground(new java.awt.Color(125, 202, 233));
    jButton3.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton3.setForeground(new java.awt.Color(13, 14, 57));
    jButton3.setText("Изменить данные");
    jButton3.setBorder(null);
    jButton3.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton3ActionPerformed(evt);
        }
    });

    jButton1.setBackground(new java.awt.Color(125, 202, 233));
    jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
    jButton1.setForeground(new java.awt.Color(13, 14, 57));
    jButton1.setText("Начать опрос");
    jButton1.setBorder(null);
    jButton1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton1ActionPerformed(evt);
        }
    });

    jButton4.setBackground(new java.awt.Color(125, 202, 233));

```



```

jButton4.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
jButton4.setForeground(new java.awt.Color(13, 14, 57));
jButton4.setText("Посмотреть историю");
jButton4.setBorder(null);
jButton4.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton4ActionPerformed(evt);
    }
});

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(22, 22, 22)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING,
false)
        .addComponent(jButton2,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
        .addComponent(jButton3,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
        .addComponent(jButton1,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
        .addComponent(jButton4,
javax.swing.GroupLayout.DEFAULT_SIZE, 129, Short.MAX_VALUE))
        .addGap(462, 462, 462)
    );
layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(50, 50, 50)
        .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 50, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(28, 28, 28)
        .addComponent(jButton3,
javax.swing.GroupLayout.PREFERRED_SIZE, 50, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(29, 29, 29)
        .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 50, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(28, 28, 28)

```

```

        .addComponent(jButton4,
javax.swing.GroupLayout.PREFERRED_SIZE, 50, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addContainerGap(50, Short.MAX_VALUE)

    );

    pack();
} // </editor-fold>

private String getLogsFromServer() {
    try (Socket socket = new Socket("localhost", 12345);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()))) {

        // Отправляем команду серверу на получение логов
        out.println("GET_LOGS");

        // Читаем ответ от сервера
        StringBuilder logs = new StringBuilder();
        String line;
        while ((line = in.readLine()) != null) {
            logs.append(line).append("\n");
        }
        return logs.length() > 0 ? logs.toString() : null;

    } catch (IOException e) {
        JOptionPane.showMessageDialog(this,
            "Ошибка подключения к серверу: " + e.getMessage(),
            "Ошибка сети",
            JOptionPane.ERROR_MESSAGE);
        return null;
    }
}

private Rule parseRule(String line) {
    try {
        line = line.substring(line.indexOf("(") + 1,
line.lastIndexOf(")")); // Убираем "rule(" и ")"
        String[] parts = line.split(";\\s*"); // Разделяем по ";",
учитывая пробелы

        int id = Integer.parseInt(parts[0].trim());
        String category = parts[1].trim();
        String name = parts[2].trim();

        // Убираем квадратные скобки у списка условий

```

```

String conditionsString = parts[3].replace("[", "").replace("]",
"".trim());

List<Integer> conditionIds = new ArrayList<>();
if (!conditionsString.isEmpty()) {
    String[] condArray = conditionsString.split(",\\s*");
    for (String cond : condArray) {
        conditionIds.add(Integer.parseInt(cond.trim()));
    }
}

return new Rule(id, category, name, conditionIds);
} catch (Exception e) {
    System.out.println("Ошибка обработки правила: " + line);
    return null;
}
}

private Condition parseCondition(String line) {
    try {
        line = line.substring(line.indexOf("(") + 1, line.indexOf(")"));
        String[] parts = line.split(";\\s*"); // Учитываем пробел или его
отсутствие

        int id = Integer.parseInt(parts[0]);
        String name = parts[1];
        return new Condition(id, name);
    } catch (Exception e) {
        System.out.println("Ошибка обработки условия: " + line);
        return null;
    }
}

private Ask parseAsk(String line) {
    try {
        line = line.substring(line.indexOf("(") + 1, line.indexOf(")"));
// Убираем "ask(" и ")"
        String[] parts = line.split(";\\s*"); // Разделяем по ";"
        int id = Integer.parseInt(parts[0].trim());
        String question = parts[1].trim();

        // Разбираем список ID условий
        List<Integer> conditionIds = new ArrayList<>();
        String conditionsString = parts[2].replace("[", "").replace("]",
"".trim());

        if (!conditionsString.isEmpty()) {
            String[] condArray = conditionsString.split(",\\s*");

```

```

        for (String cond : condArray) {
            conditionIds.add(Integer.parseInt(cond.trim()));
        }
    }
    return new Ask(id, question, conditionIds);

} catch (Exception e) {
    System.out.println("Ошибка обработки вопроса: " + line);
    return null;
}
}

private void printKnowledgeBase() {
    System.out.println("\nЗагруженные правила:");
    for (Rule rule : rules) {
        System.out.println(rule);
    }

    System.out.println("\nЗагруженные условия:");
    for (Condition cond : conditions) {
        System.out.println(cond);
    }

    System.out.println("\nЗагруженные вопросы:");
    for (Ask ask : asks) {
        System.out.println(ask);
    }
}

public List<Rule> getRules() {
    return this.rules; // Предполагаем, что `rules` – это список правил
}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Выберите файл базы знаний");

    int userSelection = fileChooser.showOpenDialog(this);

    if (userSelection == JFileChooser.APPROVE_OPTION) {
        File selectedFile = fileChooser.getSelectedFile();
    }
}

```

```

filename = selectedFile.getName();
filepath = selectedFile.getAbsolutePath();
System.out.println("File name: " + filename);
System.out.println("File path: " + filepath);

rules.clear();
conditions.clear();
asks.clear();

```

```

try (BufferedReader br = new BufferedReader(new
FileReader(selectedFile))) {
    String line;
    List<String> askLines = new ArrayList<>(); // Сохраняем
временно строки ask, чтобы обработать их после cond

    while ((line = br.readLine()) != null) {
        line = line.trim();
        if (line.isEmpty()) continue;

        if (line.startsWith("rule")) {
            Rule rule = parseRule(line);
            if (rule != null) {
                rules.add(rule);
            }
        } else if (line.startsWith("cond")) {
            Condition cond = parseCondition(line);
            if (cond != null) {
                conditions.add(cond);
            }
        } else if (line.startsWith("ask")) {
            // Сохраняем, но не разбираем пока
            askLines.add(line);
        }
    }

    // Обрабатываем ask-после загрузки всех cond
    for (String askLine : askLines) {
        Ask ask = parseAsk(askLine);
        if (ask != null) {
            boolean hasCondition = false;
            for (int conditionId : ask.getConditionIds()) {
                for (Condition condition : conditions) {
                    if (condition.getId() == conditionId) {
                        hasCondition = true;

```

```

        break;
    }
}
if (hasCondition) break; // хотя бы одно условие
есть — достаточно
}

if (hasCondition) {
    asks.add(ask);
} else {
    System.out.println("Пропущен вопрос без известных
утверждений: " + ask.getQuestion());
}
}
}

// Сообщение об успехе
JOptionPane.showMessageDialog(this,
    "База знаний успешно загружена",
    "Загрузка завершена",
    JOptionPane.INFORMATION_MESSAGE);

} catch (IOException e) {
    JOptionPane.showMessageDialog(this,
        "Ошибка при чтении файла!",
        "Ошибка",
        JOptionPane.ERROR_MESSAGE);
}
}

private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    Edit editWindow = new Edit(rules, conditions, asks, filename,
filepath);
    editWindow.setVisible(true);
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    if (asks.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Данные не загружены!",
"Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    Survey survey = new Survey(asks, getRules(), this);

```

```

        survey.setVisible(true);

    }

    private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
        String logs = getLogsFromServer();
        if (logs != null) {
            LogFrame logFrame = new LogFrame();
            logFrame.setLogText(logs);
            logFrame.setVisible(true);
        }
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
         *
         * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {

            java.util.logging.Logger.getLogger(FPSKSlabal.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);

        } catch (InstantiationException ex) {

            java.util.logging.Logger.getLogger(FPSKSlabal.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);

        } catch (IllegalAccessException ex) {

            java.util.logging.Logger.getLogger(FPSKSlabal.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);

        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

```

```

java.util.logging.Logger.getLogger(FPSKSlab1.class.getName()).log(java.util.loggi
ng.Level.SEVERE, null, ex);
    }
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new FPSKSlab1().setVisible(true);
        }
    });
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JButton jButton3;
private javax.swing.JButton jButton4;
// End of variables declaration
}

```


Приложение А.11. LogFrame.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java   to
edit this template
 */
package pskslabal;

import javax.swing.*.*;
import java.awt.*.*;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;

/**
 *
 * @author Вика
 */
public class LogFrame extends javax.swing.JFrame {

    private JTextArea logTextArea;
    private JScrollPane scrollPane;

    /**
     * Creates new form LogFrame
     */
    public LogFrame() {
        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
    }

    public LogFrame(String logs) {
        initComponents();
        setLogText(logs); // Устанавливаем текст при создании окна
    }

    public void setLogText(String text) {
        jTextArea1.setText(text);
        jTextArea1.setCaretPosition(0);
    }

    /**
     * This method is called from within the constructor to initialize the
form.
     * WARNING: Do NOT modify this code. The content of this method is always

```

```

    * regenerated by the Form Editor.
    */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        jScrollPane1 = new javax.swing.JScrollPane();
        jTextArea1 = new javax.swing.JTextArea();
        jButton1 = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

        jTextArea1.setEditable(false);
        jTextArea1.setBackground(new java.awt.Color(184, 240, 236));
        jTextArea1.setColumns(20);
        jTextArea1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
        jTextArea1.setForeground(new java.awt.Color(13, 14, 57));
        jTextArea1.setLineWrap(true);
        jTextArea1.setRows(5);
        jTextArea1.setWrapStyleWord(true);
        jScrollPane1.setViewportView(jTextArea1);

        jButton1.setBackground(new java.awt.Color(125, 202, 233));
        jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 12)); // NOI18N
        jButton1.setForeground(new java.awt.Color(13, 14, 57));
        jButton1.setText("Ok");
        jButton1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                jButton1ActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup()
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .add(layout.createSequentialGroup()
                            .addGap(217, 217, 217)
                            .addComponent(jButton1))
                        .add(layout.createSequentialGroup()
                            .addGroup(layout.createSequentialGroup()

```

```

        .addGap(26, 26, 26)
        .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,          459,
javax.swing.GroupLayout.PREFERRED_SIZE)))
        .addContainerGap(27, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(37, 37, 37)
            .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,          273,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(18, 18, 18)
            .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 33, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addContainerGap(20, Short.MAX_VALUE)
        );

    pack();
} // </editor-fold>

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    dispose();
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
        *
        * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
        */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    }
}

```

```

        } catch (ClassNotFoundException ex) {

java.util.logging.Logger.getLogger(LogFrame.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);

        } catch (InstantiationException ex) {

java.util.logging.Logger.getLogger(LogFrame.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);

        } catch (IllegalAccessException ex) {

java.util.logging.Logger.getLogger(LogFrame.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);

        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(LogFrame.class.getName()).log(java.util.logging
.Level.SEVERE, null, ex);
    }
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new LogFrame().setVisible(true);
    }
});
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTextArea jTextArea1;
// End of variables declaration
}

```

Приложение А.12. Result.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GuiForms/JFrame.java   to
edit this template
 */
package pskslabal;

/**
 *
 * @author Вика
 */
public class Result extends javax.swing.JFrame {

    /**
     * Creates new form Result
     */
    public Result() {
        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
    }

    // Метод для обновления текста в JLabel
    public void setResultText(String text) {
        jLabel1.setText(text);
    }

    /**
     * This method is called from within the constructor to initialize the
form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        jLabel1 = new javax.swing.JLabel();
        jButton1 = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

        jLabel1.setBackground(new java.awt.Color(184, 240, 236));
        jLabel1.setFont(new java.awt.Font("Bahnschrift", 0, 14)); // NOI18N
```

```

jLabel1.setForeground(new java.awt.Color(13, 14, 57));
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
jLabel1.setText("jLabel1");

jButton1.setBackground(new java.awt.Color(125, 202, 233));
jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 14)); // NOI18N
jButton1.setForeground(new java.awt.Color(13, 14, 57));
jButton1.setText("Ok");
jButton1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton1ActionPerformed(evt);
    }
});

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addContainerGap(17, Short.MAX_VALUE)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()

        .addComponent(jLabel1,
javax.swing.GroupLayout.PREFERRED_SIZE, 414,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(16, 16, 16))
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()

        .addComponent(jButton1)
        .addGap(186, 186, 186)))
);
layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(38, 38, 38)
        .addComponent(jLabel1,
javax.swing.GroupLayout.PREFERRED_SIZE, 160,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(18, 18, 18)
        .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 34, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addContainerGap(16, Short.MAX_VALUE))
);

```

```

        pack();
    } // </editor-fold>

    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        dispose();
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
           *
           * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
           */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {

            java.util.logging.Logger.getLogger(Result.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (InstantiationException ex) {

            java.util.logging.Logger.getLogger(Result.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (IllegalAccessException ex) {

            java.util.logging.Logger.getLogger(Result.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

            java.util.logging.Logger.getLogger(Result.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        }
    } //</editor-fold>

```

```

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new Result().setVisible(true);
            }
        });
    }

    // Variables declaration - do not modify
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    // End of variables declaration
}

```


Приложение А.13. Rule.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
this template
 */
package pskslabal;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/**
 *
 * @author Вика
 */
public class Rule {
    private int id;
    private String category;
    private String name;
    private List<Integer> conditionIds;

    public Rule(int id, String category, String name, List<Integer>
conditionIds) {
        this.id = id;
        this.category = category;
        this.name = name;
        this.conditionIds = conditionIds;
    }

    public int getId() {
        return id;
    }

    public String getCategory() {
        return category;
    }

    public String getName() {
        return name;
    }

    public List<Integer> getConditionIds() {
        return conditionIds;
    }
}
```

```

    }

    public void setConditionIds(List<Integer> conditionIds) {
        this.conditionIds = new ArrayList<>(conditionIds); // Копируем список,
чтобы избежать изменений по ссылке
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getConditionIdsAsString() {
        return conditionIds.stream()
            .map(String::valueOf) // Преобразует каждый Integer в String
            .collect(Collectors.joining(",")); // Объединяет их через запятую
    }

    @Override
    public String toString() {
        return "Rule{" +
            "id=" + id +
            ", category='" + category + '\'' +
            ", name='" + name + '\'' +
            ", conditionIds=" + conditionIds +
            '}';
    }
}

```

Приложение А.14. Survey.java (клиент)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java   to
edit this template
 */
package pskslabal;

import javax.swing.*.*;
import java.util.*.*;
import java.io.*.*;          // Для IOException
import java.net.Socket;      // Для Socket
import java.io.PrintWriter; // Для PrintWriter
import java.nio.file.*.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 *
 * @author Вика
 */
public class Survey extends javax.swing.JFrame {

    private List<Ask> asks; // Список вопросов
    private List<Rule> rules; // Список правил
    private List<Answer> userAnswers; // Список ответов пользователя
    private int currentIndex; // Текущий индекс вопроса
    private FPSKSlabal parent; // Главное окно

    public Survey() {
        this.asks = new ArrayList<>(); // Создаем пустой список вопросов
        this.rules = new ArrayList<>();
        this.userAnswers = new ArrayList<>();
        this.currentIndex = 0;
        this.parent = null;

        initComponents();
        getContentPane().setBackground(new java.awt.Color(225, 249, 247));
    }
}
```

```

/**
 * Creates new form Survey
 */
public Survey(List<Ask> asks, List<Rule> rules, FPSKSlabal parent) {
    this.asks = asks;
    this.rules = rules;
    this.userAnswers = new ArrayList<>();
    this.currentIndex = 0;
    this.parent = parent;

    initComponents();
    getContentPane().setBackground(new java.awt.Color(225, 249, 247));
    UpdateWindow();
}

private void sendResultsToServer(String resultText) {
    try (Socket socket = new Socket("localhost", 12345);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true))
    {

        // Удаляем HTML-теги и форматируем для записи
        String plainText = resultText.replace("<html>", "")
            .replace("</html>", "")
            .replace("<br>", " | ")
            .replaceAll("\\<.*?\\>", "");

        // Записываем в файл (чистый текст)
        Path path = Paths.get("results.txt");
        String timestamp =
            java.time.LocalDateTime.now().format(java.time.format.DateTimeFormatter.ofPattern(
                "yyyy-MM-dd HH:mm:ss"));
        String logEntry = timestamp + " | " + plainText;
        Files.write(path, (logEntry + "\n").getBytes(),
            StandardOpenOption.CREATE, StandardOpenOption.APPEND);

        // Отправляем на сервер (чистый текст)
        out.println(logEntry);
        System.out.println("Результаты отправлены: " + logEntry);

    } catch (IOException e) {
        System.err.println("Ошибка: " + e.getMessage());
    }
}

private void UpdateWindow() {

```

```

        if (currentIndex < asks.size()) {
            jLabel1.setText("Вопрос " + (currentIndex + 1) + " из " +
asks.size());
            jLabel2.setText(asks.get(currentIndex).getQuestion());
        } else {
            System.out.println("Ошибка: текущий индекс вышел за границы списка
asks.");
        }
    }

    private void answerButtonClicked(boolean answer) throws ExecutionException
{
        System.out.println("Answer to question number " + (currentIndex + 1)
+ ": " + (answer ? "Yes" : "No"));

        if (currentIndex < asks.size()) {
            Ask currentAsk = asks.get(currentIndex);
            // Проверяем, существует ли уже ответ на этот вопрос
            Answer userAnswer = null;
            for (Answer ans : userAnswers) {
                if (ans.getId() == currentAsk.getId()) {
                    userAnswer = ans;
                    break;
                }
            }

            // Если ответа еще нет, создаем новый
            if (userAnswer == null) {
                userAnswer = new Answer(currentAsk.getId(),
currentAsk.getQuestion(), "");
                userAnswers.add(userAnswer);
            }

            // Если ответ "Да", добавляем все conditionId этого вопроса в
userAnswer
            if (answer) {
                for (Integer conditionId : currentAsk.getConditionIds()) {
                    userAnswer.addConditionId(conditionId); // Добавляем ID,
а не заменяем
                }
            }
        }

        System.out.println("userAnswers: " + userAnswers);

        if (currentIndex + 1 < asks.size()) {
            currentIndex++;

```

```

        UpdateWindow();
    } else {
        JOptionPane.showMessageDialog(this, "Опрос завершен!", "Конец",
JOptionPane.INFORMATION_MESSAGE);
        findBestMatches(parent.getRules(), userAnswers);
        dispose(); // Закрываем окно после завершения опроса
    }
}

private void findBestMatches(List<Rule> rules, List<Answer> userAnswers)
throws ExecutionException {
    if (rules == null || rules.isEmpty()) {
        System.out.println("Ошибка: список rules пуст!");
        return;
    }

    System.out.println("Начало поиска best совпадений...");

    List<Integer> allConditionIds = getAllConditionIds();
    List<Integer> userConditionIds = getUserConditionIds(userAnswers);

    List<Map.Entry<Double, Rule>> sortedMatches =
Collections.synchronizedList(new ArrayList<>());

    // Параллельная обработка
    ExecutorService executor =
Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors());
    List<Future<?>> futures = new ArrayList<>();

    for (Rule rule : rules) {
        futures.add(executor.submit(() -> {
            double percentage = calculateMatchPercentage(rule,
userConditionIds, allConditionIds);
            if (percentage >= 72) {
                sortedMatches.add(new AbstractMap.SimpleEntry<>(percentage,
rule));
            }
        }));
    }

    // Ждём завершения всех задач
    for (Future<?> future : futures) {
        try {
            future.get();
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }
    }
}

```

```

    }

    executor.shutdown();

    // Отображение результатов
    Result resultWindow = new Result();
    resultWindow.setVisible(true);

    String timestamp =
java.time.LocalDateTime.now().format(java.time.format.DateTimeFormatter.ofPattern(
"yyyy-MM-dd HH:mm:ss"));
    StringBuilder logEntry = new StringBuilder(timestamp + " | ");

    if (sortedMatches.isEmpty()) {
        String healthyMessage = "Вы здоровы!";
        System.out.println(healthyMessage);
        resultWindow.setResultText(healthyMessage);
        logEntry.append(healthyMessage);
    } else {
        sortedMatches.sort((a, b) -> Double.compare(b.getKey(), a.getKey()));
        StringBuilder resultText = new StringBuilder("<html>");
        for (Map.Entry<Double, Rule> entry : sortedMatches) {
            String matchText = "Вероятность того, что у вас " +
entry.getValue().getName() + ": " + String.format("%.2f", entry.getKey()) + "%";
            System.out.println(matchText);
            resultText.append(matchText).append("<br>");
            logEntry.append(matchText).append(" | ");
        }
        resultText.append("</html>");
        resultWindow.setResultText(resultText.toString());
    }

    sendResultsToServer(logEntry.toString());
}

```

```

    private double calculateMatchPercentage(Rule rule, List<Integer>
userConditionIds, List<Integer> allConditionIds) {
        if (rule == null || allConditionIds.isEmpty()) {
            return 0;
        }

        List<Integer> ruleConditions = rule.getConditionIds();

```

```

        int totalConditions = allConditionIds.size();
        int matchCount = 0;

        for (Integer id : allConditionIds) {
            boolean inRule = ruleConditions.contains(id);
            boolean inAnswers = userConditionIds.contains(id);

            if (inRule == inAnswers) { // Подсчитываем совпадения (1 и 1 или
0 и 0)
                matchCount++;
            }
        }
        System.out.println(matchCount + " " + totalConditions);
        return (double) matchCount / totalConditions * 100;
    }

    private List<Integer> getAllConditionIds() {
        List<Integer> allIds = new ArrayList<>();
        for (Rule rule : rules) {
            for (Integer id : rule.getConditionIds()) {
                if (!allIds.contains(id)) {
                    allIds.add(id);
                }
            }
        }
        return allIds;
    }

    private List<Integer> getUserConditionIds(List<Answer> userAnswers) {
        List<Integer> userConditionIds = new ArrayList<>();
        for (Answer answer : userAnswers) {
            //System.out.println("CondAnswerId: " +
answer.getCondAnswerId());
            for (String idStr : answer.getCondAnswerId().split(";")) {
                idStr = idStr.trim(); //удаляем пробелы
                if (!idStr.isEmpty() && idStr.matches("\\d+")) { // Проверяем,
что строка не пустая и это число
                    try {
                        int id = Integer.parseInt(idStr);
                        if (!userConditionIds.contains(id)) {
                            userConditionIds.add(id);
                        }
                    } catch (NumberFormatException e) {
                        System.out.println("Ошибка преобразования ID: " +
idStr);
                    }
                }
            }
        }
    }

```



```

        } //else {
            //System.out.println("Некорректный ID: " + idStr);
        //}
    }
}

System.out.println("UserIdConditionList: " + userConditionIds);
return userConditionIds;
}

/**
 * This method is called from within the constructor to initialize the
form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jLabel1 = new javax.swing.JLabel();
    jLabel2 = new javax.swing.JLabel();
    jButton1 = new javax.swing.JButton();
    jButton2 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

    jLabel1.setFont(new java.awt.Font("Bahnschrift", 0, 14)); // NOI18N
    jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
    jLabel1.setText("Номер вопроса");

    jLabel2.setFont(new java.awt.Font("Bahnschrift", 0, 14)); // NOI18N
    jLabel2.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
    jLabel2.setText("Вопрос");

    jButton1.setBackground(new java.awt.Color(125, 202, 233));
    jButton1.setFont(new java.awt.Font("Bahnschrift", 0, 14)); // NOI18N
    jButton1.setText("Да");
    jButton1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton1ActionPerformed(evt);
        }
    });

    jButton2.setBackground(new java.awt.Color(125, 202, 233));

```

```

jButton2.setFont(new java.awt.Font("Bahnschrift", 0, 14)); // NOI18N
jButton2.setText("Her");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});

javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
        .addGap(139, 139, 139)
        .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 82, javax.swing.GroupLayout.PREFERRED_SIZE)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 197,
Short.MAX_VALUE)
        .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 80, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(143, 143, 143))
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
        .addComponent(jLabel2,
javax.swing.GroupLayout.PREFERRED_SIZE, 545,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(46, 46, 46))
    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()
        .addComponent(jLabel1,
javax.swing.GroupLayout.PREFERRED_SIZE, 160,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(235, 235, 235))))
);
layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addContainerGap(40, Short.MAX_VALUE)

```

```

        .addComponent(jLabel2,
javax.swing.GroupLayout.PREFERRED_SIZE, 65, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(33, 33, 33)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jButton1,
javax.swing.GroupLayout.PREFERRED_SIZE, 37, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jButton2,
javax.swing.GroupLayout.PREFERRED_SIZE, 37,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(28, 28, 28)
        .addComponent(jLabel1)
        .addGap(38, 38, 38))
    );

    pack();
} // </editor-fold>

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        answerButtonClicked(false);
    } catch (ExecutionException ex) {
        Logger.getLogger(Survey.class.getName()).log(Level.SEVERE, null,
ex);
    }
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        answerButtonClicked(true);
    } catch (ExecutionException ex) {
        Logger.getLogger(Survey.class.getName()).log(Level.SEVERE, null,
ex);
    }
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting
code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the
default look and feel.
        *
        * For details see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html

```

```

        */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {

javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {

java.util.logging.Logger.getLogger(Survey.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (InstantiationException ex) {

java.util.logging.Logger.getLogger(Survey.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (IllegalAccessException ex) {

java.util.logging.Logger.getLogger(Survey.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(Survey.class.getName()).log(java.util.logging.L
evel.SEVERE, null, ex);
        }
    }
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new Survey().setVisible(true);
    }
});
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
// End of variables declaration
}

```

Приложение А.15. SurveyServer.java (сервер)

```
/*
 *      Click      nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
this template
 */
package pskslabal;

import java.io.*;
import java.net.*;
import java.nio.file.*;

/**
 *
 * @author Вика
 */
public class SurveyServer {
    private static final int PORT = 12345;
    private static final String LOG_FILE = "results.txt";

    public static void main(String[] args) {
        System.out.println("Сервер логов запущен на порту " + PORT);

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            while (true) {
                Socket clientSocket = serverSocket.accept();
                new Thread(() -> handleClient(clientSocket)).start();
            }
        } catch (IOException e) {
            System.err.println("Ошибка сервера: " + e.getMessage());
        }
    }

    private static void handleClient(Socket clientSocket) {
        try
            (BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream())));
            PrintWriter out = new
PrintWriter(clientSocket.getOutputStream(), true)) {

            String command = in.readLine();
            if ("GET_LOGS".equals(command)) {
                sendLogFile(out);
            }
        } catch (IOException e) {
            System.err.println("Ошибка клиента: " + e.getMessage());
        }
    }
}
```

```

    } finally {
        try {
            clientSocket.close();
        } catch (IOException e) {
            System.err.println("Ошибка закрытия сокета: " +
e.getMessage());
        }
    }
}

private static void sendLogFile(PrintWriter out) {
    try {
        // Полный путь к файлу в папке проекта
        String projectDir = System.getProperty("user.dir");
        Path path = Paths.get(projectDir, "results.txt");

        System.out.println("Ищу файл по пути: " + path); // Для отладки

        if (Files.exists(path)) {
            System.out.println("Файл найден, отправляю содержимое");
            String content = Files.readString(path);
            out.println(content);
        } else {
            System.out.println("Файл не найден");
            out.println("Файл логов не найден по пути: " + path);
        }
    } catch (IOException e) {
        System.err.println("Ошибка при работе с файлом: " +
e.getMessage());
        out.println("Ошибка сервера при чтении логов");
    }
}
}

```

Приложение В. UML – диаграммы

Приложение В.1. UML-диаграмма последовательности

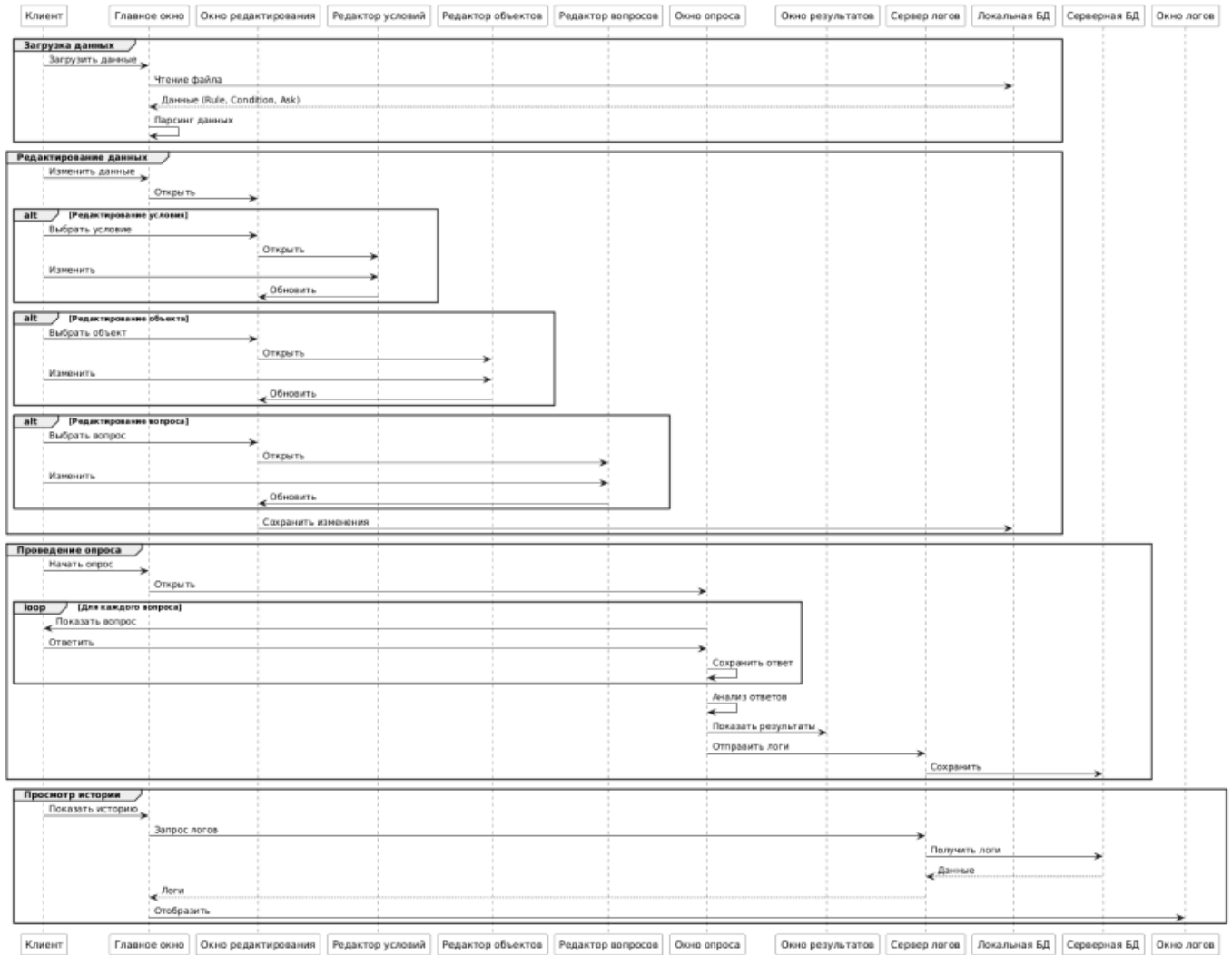


Рисунок 13 – UML-диаграмма последовательности

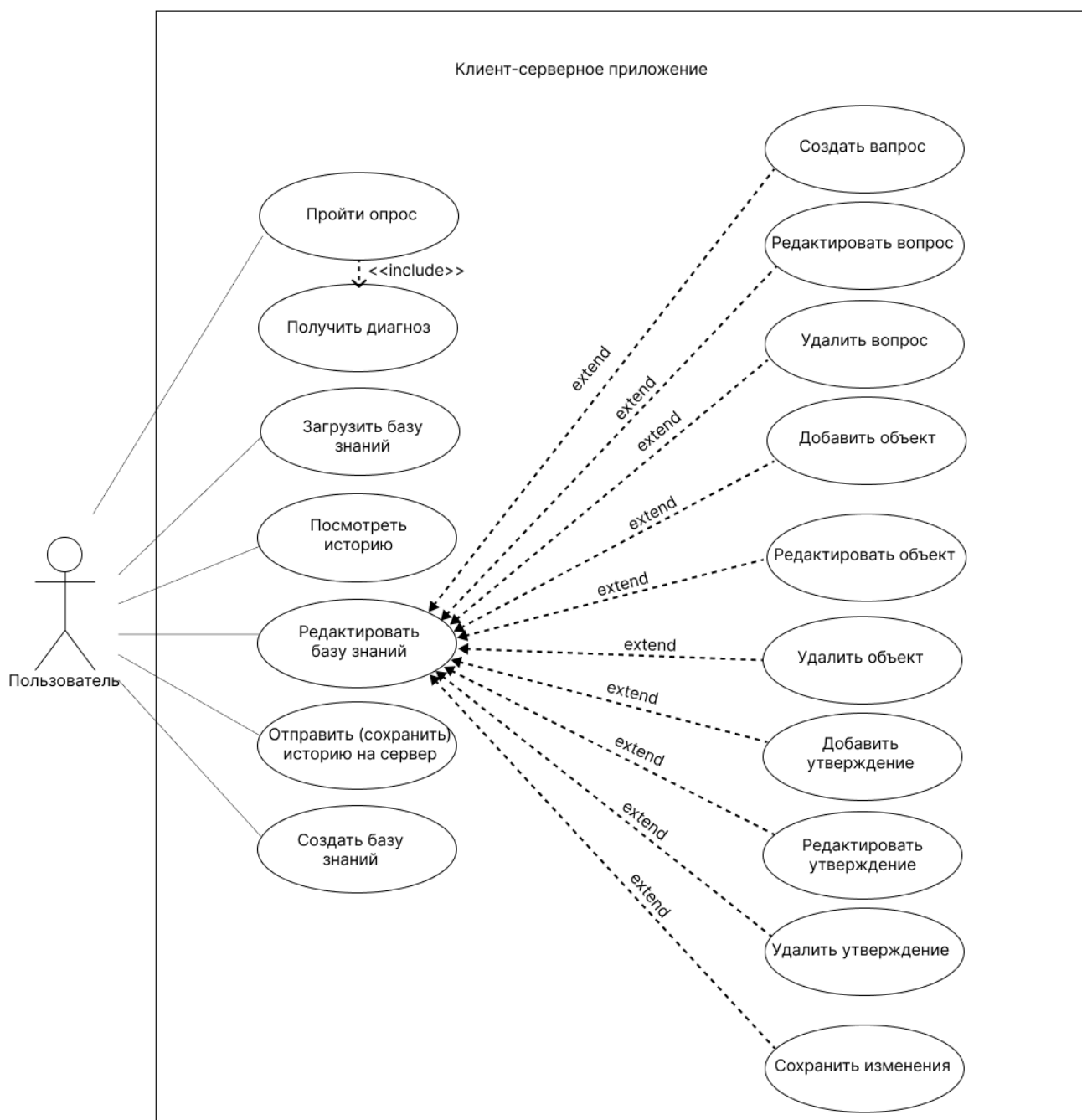


Рисунок 14 – UML-диаграмма вариантов использования приложения

Приложение В.3. UML- диаграмма развертывания

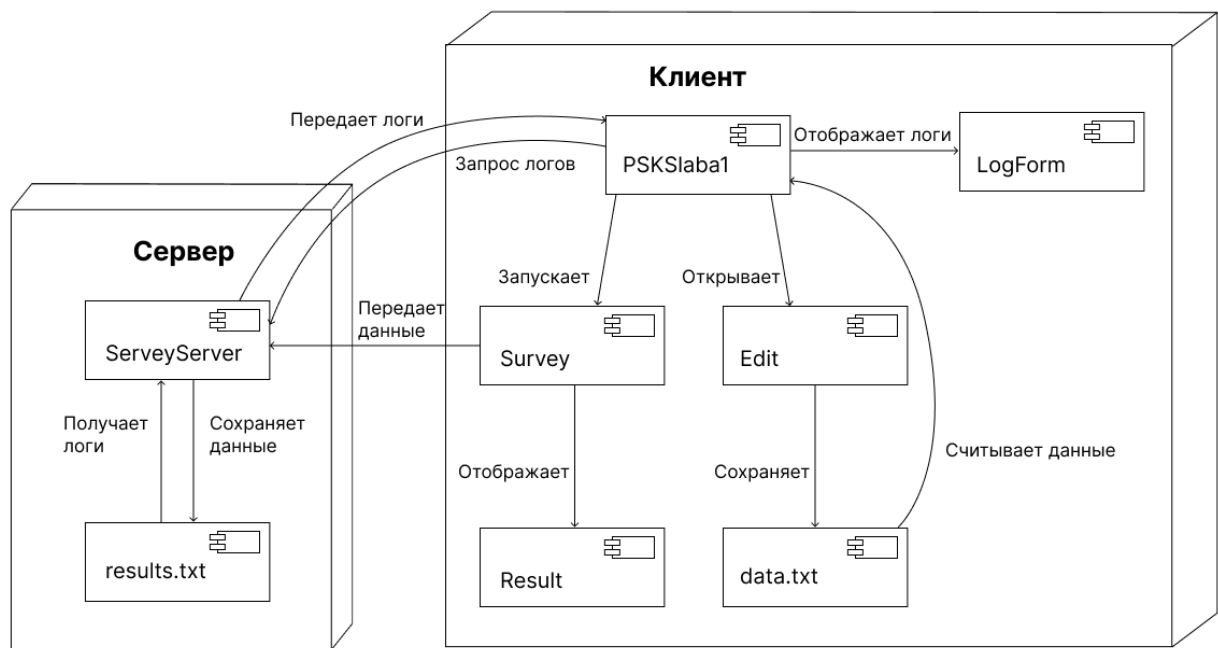


Рисунок 15 – UML-диаграмма развертывания

Приложение В.4. UML- диаграмма деятельности

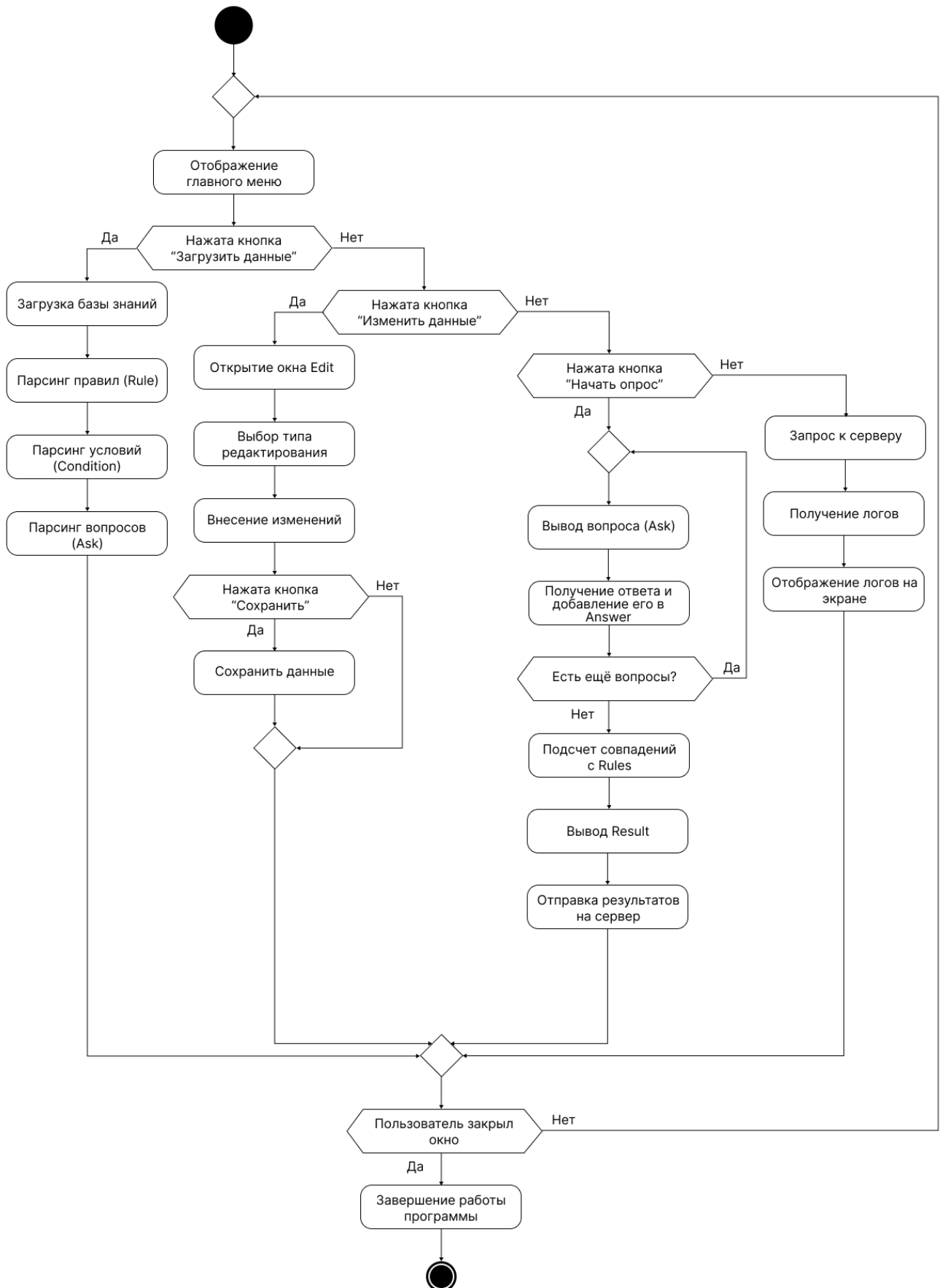


Рисунок 16 – UML-диаграмма деятельности

Приложение В.4. UML-диаграмма классов

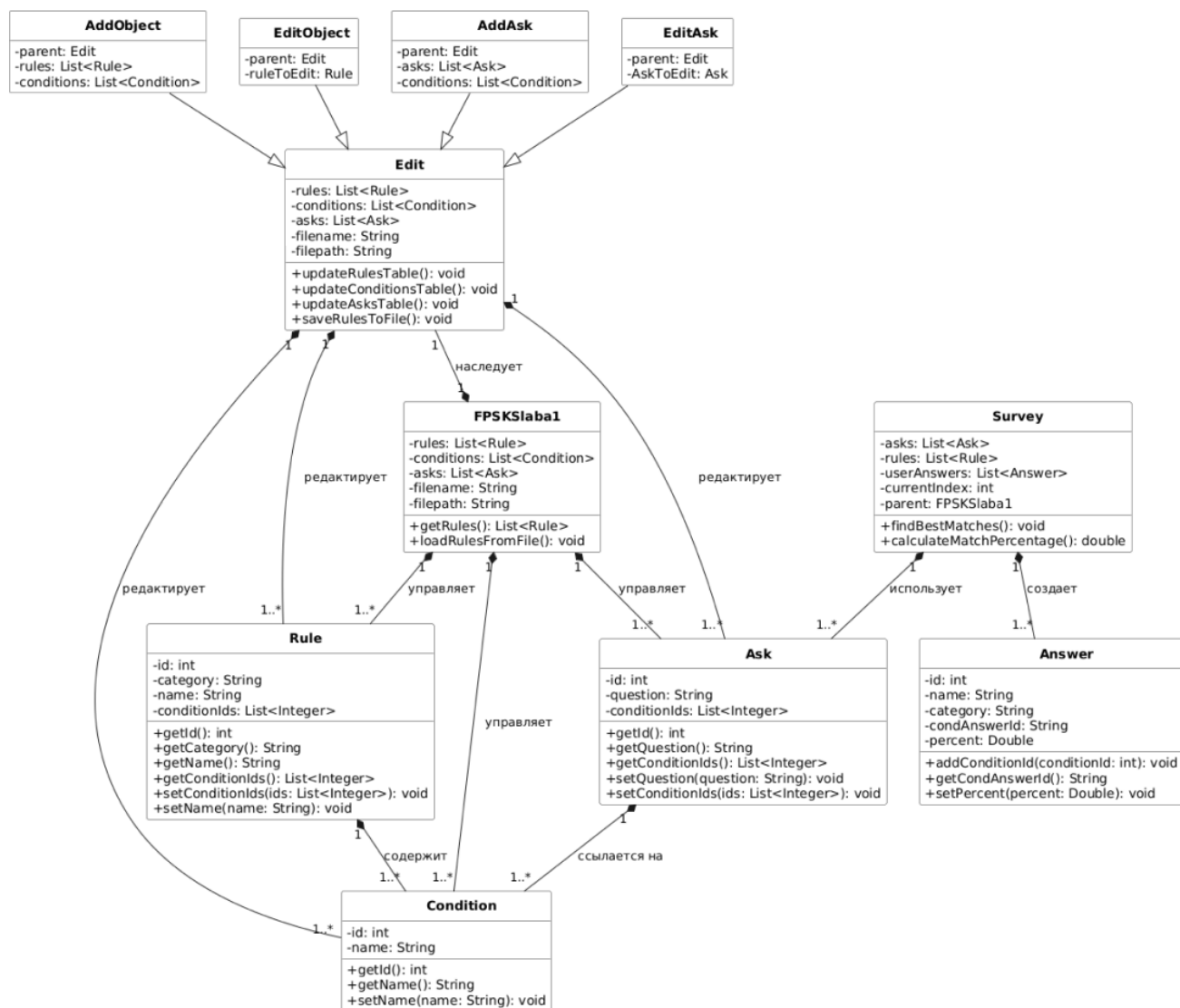


Рисунок 17 – UML-диаграмма классов